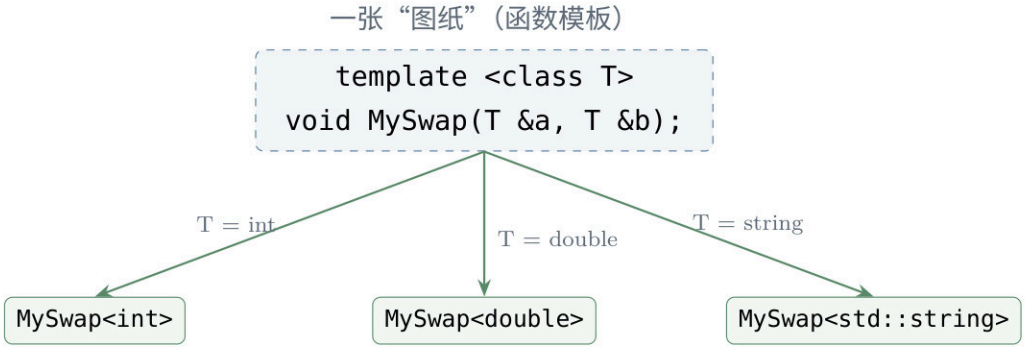
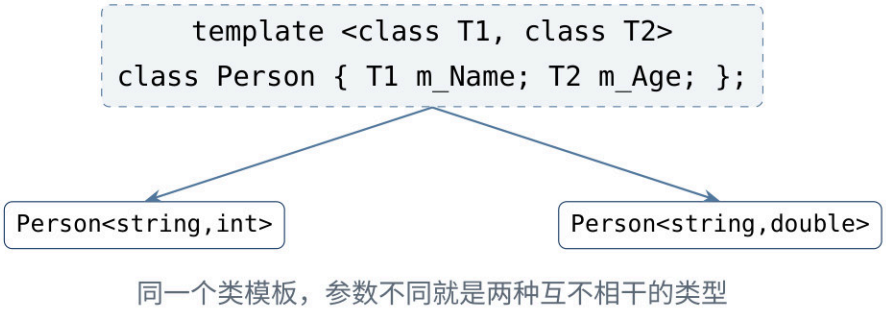
C++ 模板

函数模板与类模板 ——可视化专题讲义

一份写给第一次学模板的你：从“为什么要写两遍 Swap”讲到“为什么分文件会链接失败”





XeLaTeX 排版·A4

正式练习 38 题（25 + 10 + 3）

适用：预习·课堂辅助·复习·考前巩固

目录

第 1 章 导读：这份讲义怎么读 **1**

阅读说明 1

1.1 本专题学什么 1

1.2 在课程体系中的位置 1

1.3 前置知识地图 2

1.4 原始资料内容重新编码 2

1.5 原始资料纠错 3

1.6 知识点总览与关系图 5

第 2 章 学科小史与现实应用 **5**

2.1 学科小史：模板是怎样出现的? 5

2.2 从课本到现实：模板到底有什么用? 6

第 3 章 函数模板 **7**

3.1 K1 为什么需要模板：重复代码里藏着一个“空格” 7

3.2 K2 函数模板的语法 11

3.3 K3 模板实例化：从“图纸”到真正的函数 14

3.4 K4 两种使用方式：自动类型推导与显式指定类型 17

3.5 K5 模板对类型的隐含要求 20

第 4 章 类模板 **24**

4.1 K6 类模板的语法与实例化 24

4.2 K7 类模板对象作函数参数 28

4.3 K8 类模板成员函数的类外实现 30

4.4 K9 成员函数“用到才实例化” 33

第 5 章 类模板的分文件编写 **36**

5.1 K10 为什么分文件会出现链接错误 36

5.2 三种解决方法 38

5.3 走向数据结构：第一个通用容器 `MyStack<T>` 40

第 6 章 正式练习（38 题） **42**

使用说明与知识点—题目对应表 42

6.1 A. 基础巩固训练（25 题） 42

6.2 B. 中等偏下提升训练（10 题） 48

6.3 C. 跨学科迁移训练（3 题） 52

第 7 章 答案速查 **53**

第 8 章 详细解析 **54**

8.1 基础单选题解析（第 1–10 题） 54
8.2 基础多选题解析（第 11–15 题） 57
8.3 基础填空题解析（第 16–20 题） 59
8.4 基础解答题解析（第 21–25 题） 60
8.5 提升题解析（第 26–35 题） 62
8.6 跨学科迁移题解析（第 36–38 题） 66

第 9 章 总复习与参考资料 **68**

9.1 高频易错点总表 68
9.2 基础解题步骤总结 69
9.3 相近概念总对比 69
9.4 跨学科 / 相似知识总对比表 70
9.5 后续课程连接 70
9.6 参考资料 70

第 1 章　导读：这份讲义怎么读

阅读说明

这份讲义根据你在“C++ 前传：类与对象 / 封装 / 模板”文件夹中的 5 份练习源文件整理而成：函数模板.cpp、类模板.cpp、类模板成员函数类外实现.cpp、类模板分文件编写/person.h 与 类模板分文件编写/类模板分文件.cpp。源文件决定了“讲什么”；为了让第一次接触模板的同学不断链，讲义还补上了源文件里没写出来、但理解时绕不开的内容。全书用固定的颜色和标签区分内容的来源与用途：

标记 / 颜色	含义
【原始资料】	直接来自你的源文件，经过逐行核对
【教学补全】	源文件没有写出、由讲义依据 C++ 标准资料补充的内容（已核验）
【拓展联想】	只用来建立联系，当前阶段不要求掌握
蓝色框	定义、核心语法、规则：必须掌握的知识
浅红框	易错点、原图纠错
米色框	例题、生活中的它、学科小史、科普
绿色框	总结、方法、自检
✔ △ ✘	对照模块中的迁移判断：直接成立 / 加条件成立 / 不成立

所有代码都在 g++ 13（-std=c++17）上实际编译运行过；凡写“报错为……”的地方，引用的都是真实的编译器输出（为便于阅读，略去了文件名和行号）。你在 Windows 上用的 MinGW g++ 输出措辞基本相同。

1.1　本专题学什么

一句话概括：**模板让你把“只有类型不同、逻辑完全相同”的代码只写一遍，由编译器按需生成具体版本**。本专题依次回答五个问题：

1. 为什么写了 SwapInt 还要写 SwapDouble？能不能只写一个？（函数模板）
2. 编译器拿到模板后到底做了什么？“T 是什么”由谁决定？（实例化与类型推导）
3. 类能不能也“空着类型”？Person<string,int> 里的尖括号是什么意思？（类模板）
4. 成员函数写到类外面时，为什么每个函数前都要再写一遍 template？（类外实现）
5. 为什么把类模板拆成 .h + .cpp 就会出现 undefined reference？（实例化时机与分文件）

1.2　在课程体系中的位置

你所在的文件夹路径是“C++ 前传→类与对象→封装→模板”。模板是“类与对象”通往“数据结构”的那座桥：数据结构课里的顺序表、链表、栈、队列、二叉树，几乎都要写成“元素类型待定”的类模板，C++ 标准库里的 vector、stack、queue 本身也都是类模板。

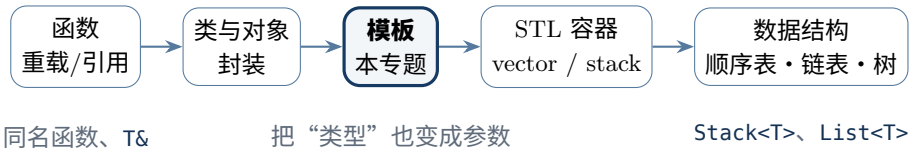


图 1　模板在“C++ 前传→数据结构”路线中的位置

图 1 从左往右读：左侧是学模板之前必须会的东西（函数重载、引用、类与封装），右侧是模板马上要服务的对象。箭头表示“后者要用到前者”。

1.3　前置知识地图

模板本身的新东西并不多，难点主要出在“前置知识不牢”。下图列出本专题真正要用到的前置知识；若某一项不熟，先回去翻一下对应内容，再读正文。

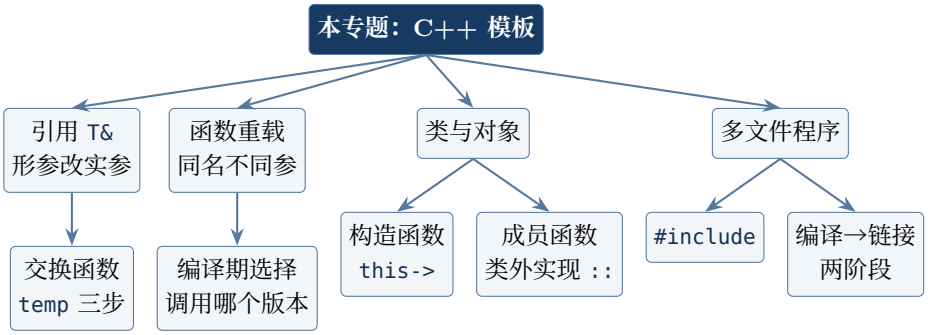


图 2　前置知识地图：箭头从“要用到的知识”指向“它依赖的更基础知识”

教学补全 编译与链接，30 秒回顾

一个多文件程序要经历两步。**编译**：编译器一次只看一个 `.cpp` 文件（连同它 `#include` 进来的头文件，合称一个“编译单元”），把它翻译成目标文件 `.o`；遇到“只有声明、没有定义”的函数，就在 `.o` 里记一笔“这里需要一个外部符号”。**链接**：链接器把所有 `.o` 拼在一起，按名字为每个“外部符号”找到唯一的定义；找不到就报 `undefined reference`。第 5 章的分文件问题完全建立在这两句话上。

1.4　原始资料内容重新编码

5 份源文件都是纯文本代码，全部内容都能可靠识别，**没有无法辨认之处**。下表按“知识点”重新编排了每份文件说了什么（不是逐行照抄）。

源文件	实际讲到的内容	本讲义位置
函数模板.cpp	用 SwapInt/SwapDouble 的重复引出模板；语法 template <typename T>；两个类型参数 T、M；两种用法：自动类型推导 MySwap(a,b,c)、显式指定 MySwap<double,int>(f,d,e)；总结“提高复用性”	3.1–3.5
类模板.cpp	类模板语法 template <class NameType, class AgeType>；Person<string,int> p(" 张三",20)；以 Person<string,int>& 作函数参数	4.1–4.2
类模板成员函数类外实现.cpp	构造函数与 ShowPerson 在类外实现：每个函数前重写 template <class T1,class T2>，并用 Person<T1,T2>:: 限定	4.3
person.h	只有声明的类模板头文件（含 #pragma once）	5.2
类模板分文件.cpp	分文件导致链接错误的完整示例与原因分析；两种解决方法：包含 .cpp、合并为 .hpp；“孤立 template 行”易错点	5.1–5.3

1.5　原始资料纠错

源文件整体正确，程序都能编译运行。下面 6 处属于“笔误、不严谨或与实测不符”，按规范显式列出，不静默修改。

原图纠错 1：错别字

原始写法：函数模板.cpp 开头注释“其函数返回值类型和形参类型可以不具体**制定**”。

存在的问题：“制定”指拟定规则、计划；这里要表达的是“明确规定是哪一种”，应为“指定”。

规范写法：“……可以不具体**指定**，用一个虚拟的类型来代表”。　**影响**：不影响后续内容。

原图纠错 2：MySwap 里混入了与“交换”无关、且会截断数据的语句

原始写法：MySwap(T&a, T&b, M&c) 中除交换外还有 c = 1.1; double m = a + b + c;，并以 MySwap<double,int>(f,d,e) 调用。

存在的问题：第二次调用时 M = int，语句 c = 1.1 把 double 赋给 int，**小数部分被截断**，e 的值是 1 而不是 1.1，于是 m = 2.2 + 1.1 + 1 = 4.3。程序不报错，但用 -Wconversion 编译会得到警告：conversion from 'double' to 'int' changes value from '1.1' to '1'。这正好说明一个模板要点：**模板函数体对每一种可能的类型都要“说得通”**，否则就会出现编译错误或这种静默的数据损失（见 3.5 节）。

规范写法：演示两种调用方式时，让 MySwap 只做交换：template<class T> void MySwap(T&a, T&b)；若要演示“两个类型参数”，另写一个语义清楚的例子（见 3.2 节）。　**影响**：输出 m = 31.1（第一次）与 m = 4.3（第二次）均已实测核对。

原图纠错 3：注释写 typename，代码用 class

原始写法：类模板.cpp 注释解释“typename 是声明一个虚拟类型”，而代码写的是 template < class NameType, class AgeType>。

存在的问题：不是错误，但初学者会以为二者不同。在**模板参数列表里**，`typename` 与 `class` 完全等价（3.2 节）。

规范写法：注释补一句“这里写 `class` 与写 `typename` 含义相同”。　**影响**：无。

原图纠错 4：头文件中缺少 `<string>`，并使用了 `using namespace std`

原始写法：`person.h` 与 `类模板.cpp` 使用了 `string`，但只包含 `<iostream>`；`person.h` 中写有 `using namespace std;`。

存在的问题：`<iostream>` 是否顺带包含 `<string>` 由编译器实现决定，标准不保证，换个编译器可能编译失败；头文件里的 `using namespace std;` 会强加给所有包含它的文件，容易引发命名冲突。

规范写法：使用 `string` 就显式 `#include <string>`；头文件里写 `std::string`、`std::cout`。　**影响**：不影响本机运行结果。

原图纠错 5：“若仍把 person.cpp 一起编译，可能出现重复定义”与实测不符

原始写法：`类模板分文件.cpp` 注释：“`#include "person.cpp"` 后，若仍把 `person.cpp` 一起编译，可能出现重复定义”。

存在的问题：实测 `g++ main.cpp person.cpp` 可以正常链接运行。原因是：模板成员函数实例化出来的代码属于“可在多个编译单元中重复出现”的一类（`nm` 显示为弱符号 `W`），链接器会自动合并，不算重复定义。**只有**当 `person.cpp` 里还写了普通的（非模板、非 `inline`）函数时，才会真的出现 `multiple definition`。

规范写法：“若 `person.cpp` 只含模板定义，一起编译通常不会出错，但它被编译了两次；一旦其中加入普通函数定义就会重复定义，所以不推荐这种写法。”　**影响**：结论“方法 1 不常用、推荐 `.hpp`”不变。

原图纠错 6：两处措辞需要更精确

原始写法①：“类模板中成员函数创建时机是在**调用阶段**”。**问题**：“调用阶段”容易被理解成“程序运行、真正调用时”。实际上实例化发生在**编译阶段**：编译器在某个编译单元里看到这个成员被使用时才生成它（4.4 节）。

原始写法②：“孤立的 `template` 行会接到 `main` 上，`main` 变成函数模板→ **编译/链接**报错”。**问题**：实测在**编译阶段**就报错：`error: cannot declare '::main' to be a template`，到不了链接。

规范写法：①改为“编译期、在用到时才实例化”；②改为“编译报错”。　**影响**：结论方向不变，只是更精确。

1.6　知识点总览与关系图

全书 10 个核心知识点及其依赖关系如下。实线箭头表示“理解后者需要前者”，虚线箭头表示“后者解决了前者留下的问题”。

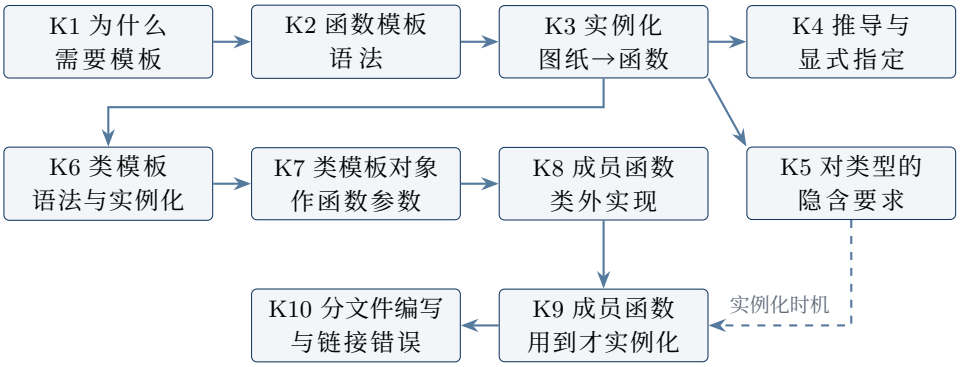


图 3　10 个核心知识点的关系图

图 3 的主干是 K1 → K3：先明白“为什么要模板”，再理解“编译器拿模板做了什么”。理解了实例化（K3），函数模板的用法（K4、K5）和类模板（K6–K8）就都是同一件事的不同面貌；而 K9、K10 只是把“实例化发生在何时、何处”这个问题追问到底。

到这里，我们已经解决了什么？下一步呢？

我们知道了要学什么、要用到哪些旧知识、源文件里哪些地方需要修正。但还有一个问题没回答：**模板是凭空设计出来的吗？**下一章先花两页看看它从哪里来、在真实软件里待在哪里，然后正式开讲。

第 2 章　学科小史与现实应用

2.1　学科小史：模板是怎样出现的？

当时遇到的问题：容器类写不完

20 世纪 80 年代中期，C++ 已经有了类，但还没有模板。程序员想写一个“向量”或“链表”时，只有三条路：为 `int`、`double`、`Person`……各写一份几乎相同的类；用 C 语言的宏做文本替换；或者让容器统一存放“指向公共基类的指针”，取出时再强制转换。第一条路代码成倍膨胀，第二条路没有类型检查、出错信息难懂，第三条路把类型错误推迟到运行时。C++ 的设计者 Bjarne Stroustrup 在 1988 年的论文《Parameterized Types for C++》中把目标说得很直接：要能写出“元素类型留作参数”的通用容器，例如 list、vector、关联数组。

模板并不是 C++ 的独创。“把类型当参数”的思想此前已在别的语言里出现：David Musser 在 1971 年前后就提出了泛型编程的部分想法；1983 年标准化的 Ada 是第一门支持“泛型”的主流语言；1985 年的 Eiffel 是第一门内置泛型类的面向对象语言。C++ 的特点是把这个想法做成了**编译期生成代码**，所以生成的代码和手写一样快。



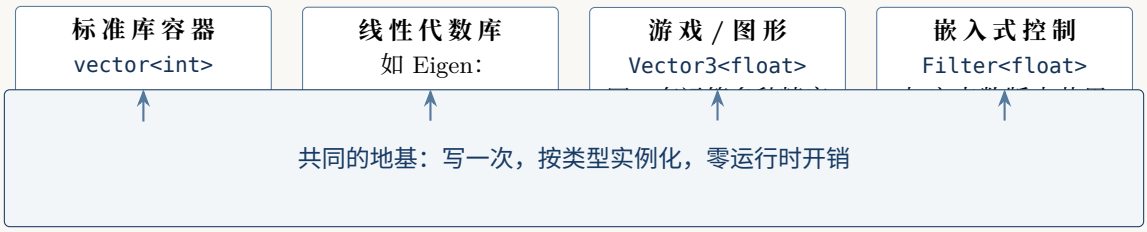
图 4　模板发展时间轴（年份采用可靠来源中的保守表述，见参考资料）

时间轴上有两个节点和本专题直接相关。第一个是 **STL**：Alexander Stepanov 与 Meng Lee 设计的标准模板库在 1993 年 11 月被介绍给 C++ 标准委员会，1994 年 7 月获得最终批准，从此 `vector<int>` 成了每个 C++ 程序员的日常。第二个是 **`export`**：C++98 曾引入关键字 `export`，本意是让模板也能“声明放 `.h`、实现放 `.cpp`”分开编译——也就是第 5 章那个链接错误的“官方解法”。可惜它极难实现，几乎只有 EDG 前端做到了，最终在 C++11 中被删除。所以今天我们仍然把类模板写在 `.hpp` 里，这背后正是一段失败的历史。

为什么今天仍然要学模板？因为它是 C++ 标准库、各类数值计算与图形库的地基；而且“把一段逻辑对一整类对象写一次”这种思想，在你正在学的高等代数（对任意数域 P 证明一次定理）和数理统计（对整个分布族讨论一次估计量）里也处处可见（3.1 节、4.1 节的对照）。

2.2　从课本到现实：模板到底有什么用？

模板在真实软件里的位置



①**标准库容器**。你在数据结构课里会用到的 `vector`、`list`、`stack`、`queue`、`map`、`sort`，全都是模板。②**线性代数库**。科学计算库 Eigen 把矩阵的元素类型和尺寸都作为模板参数，同一份代码可以算 `float` 矩阵、`double` 矩阵甚至复数矩阵，而且尺寸已知时编译器能把小矩阵运算完全展开。③**游戏与图形**。三维向量、颜色、包围盒等小类型常写成 `Vector3<T>`，在显卡上用 `float`，在物理模拟里用 `double`。④**嵌入式与控制**。同一个数字滤波器算法可以实例化成 `float` 版和定点数版，分别跑在有、无浮点运算单元的单片机上。

代价也要知道：每种类型都会生成一份代码，用得多了可执行文件会变大（俗称“代码膨胀”），编译也会变慢；模板出错时编译器信息往往很长。C++20 的 Concepts 正是为了让这类错误信息变得好懂（3.5 节拓展）。

到这里，我们已经解决了什么？下一步呢？

我们知道了模板为解决“容器和算法要为每种类型重写一遍”而生，并且它今天就在你将要使用的每一个 STL 容器里。下一章从你写过的那两个交换函数出发，正式学习函数模板。

第 3 章　函数模板

上一章说到，模板为“同一段逻辑要为每种类型重写一遍”而生。本章就从你在 函数模板.cpp 里亲手写过、又亲手注释掉的两个交换函数讲起，依次回答：为什么需要模板（3.1）、怎么写（3.2）、编译器拿它做什么（3.3）、怎么调用（3.4）、它对类型有什么要求（3.5）。

本章回答三个问题：①函数模板解决了什么问题？②编译器如何把模板变成真正的函数？③什么情况下模板调用会失败？　**知识路线：**重复代码 → 类型参数 → 实例化 → 推导/显式指定 → 类型要求。

3.1　K1　为什么需要模板：重复代码里藏着一个“空格”

学科坐标：C++ 程序设计·泛型编程入门　前置：函数、引用、函数重载　后续：类模板、STL、数据结构中的通用容器　相关课程：高等代数（数域上的一般理论）

它解决什么问题　在 函数模板.cpp 里，你先写了交换两个 int 的 SwapInt，又写了交换两个 double 的 SwapDouble，并在注释中写下了关键观察：【原始资料】“发现以上两个函数除了类型不一样，函数本身逻辑基本一样。”如果还要交换 char、string、Person，就得继续复制、粘贴、改类型。问题是：**能不能把“类型”也空出来，像函数参数一样，用的时候再填？**

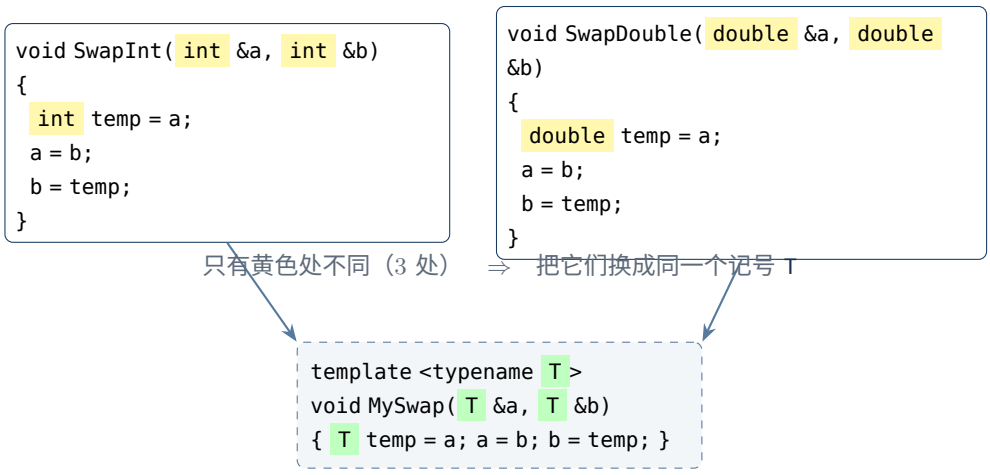


图 5　两个交换函数逐行对比：差异只在类型，其余逐字相同

图 5 的上半部分把两个函数并排放：黄色标出的 3 处是仅有的差异，而且这 3 处总是“同一个类型”。下半部分把它们统一替换成记号 T（绿色），就得到一个“缺一个类型”的函数，这就是函数模板。

> **一句话直观理解**　函数模板 = 把函数里“会变的类型”挖成一个空格 `T`，函数逻辑只写一次；用的时候往空格里填具体类型。

定义 泛型编程与函数模板

泛型编程（generic programming）：把与具体类型无关的算法逻辑只写一次，把类型作为参数，使同一份代码适用于多种类型的编程方式。

函数模板（function template）：带有类型参数的函数“蓝图”。它本身不是一个可以直接调用的函数；只有给定了类型参数之后，编译器才据此生成真正的函数。【原始资料】源文件的说法：“建立一个通用函数，其函数返回值类型和形参类型可以不具体指定，用一个虚拟的类型来代表”，目的是“提高代码的复用性”。

推导 / 论证 为什么两个函数可以合并成一个模板

要说明什么：`SwapInt` 与 `SwapDouble` 可以由同一份带类型参数的代码表示。

已知：两函数的代码见图 5。

思路：如果把一个函数里所有的 `int` 都替换成 `double` 后，恰好得到另一个函数，那它们就是“同一份代码 + 不同的类型”。

逐步：①把 `SwapInt` 中的 3 处 `int` 全部替换为 `double`（函数名除外），得到 `void f(double &a, double &b){double temp=a; a=b; b=temp;}`；②逐字比较，它与 `SwapDouble` 的函数体完全一致；③交换算法“`temp` 存 $a \to b$ 赋给 $a \to$ `temp` 赋给 b”中，没有任何一步依赖“a 是整数”这件事，它只用到了“能复制、能赋值”。

结论：用一个记号 `T` 代替这 3 处类型，得到的 `MySwap` 在 `T=int` 时就是 `SwapInt`，在 `T=double` 时就是 `SwapDouble`。□

一句话：能合并的前提是“逻辑不依赖具体类型”；第 3.5 节会看到，当逻辑用到了某种类型才有的操作时，模板就会对类型提出要求。

例题 1　交换 `char` 还要改几处？

已知：已有 `SwapInt`。**求**：不用模板时，写出交换 `char` 的版本需要修改几处类型；用模板时又需要写几行新代码？

第一步：数 `SwapInt` 中的类型位置：两个形参 + `temp`，共 3 处，都改成 `char`，再改函数名，共 4 处修改、整整一个新函数。

第二步：有了 `MySwap` 模板，直接写 `MySwap(c1, c2);`，**新代码 0 行**，编译器会自己生成 `char` 版本。

检查：若有 n 种类型，手写需要 n 个函数，模板只需 1 个（下图左）。

本题想说明：模板省掉的是“人写的代码”，不是“机器生成的代码”——机器仍然为每种类型生成一份（下图右，详见 3.3）。

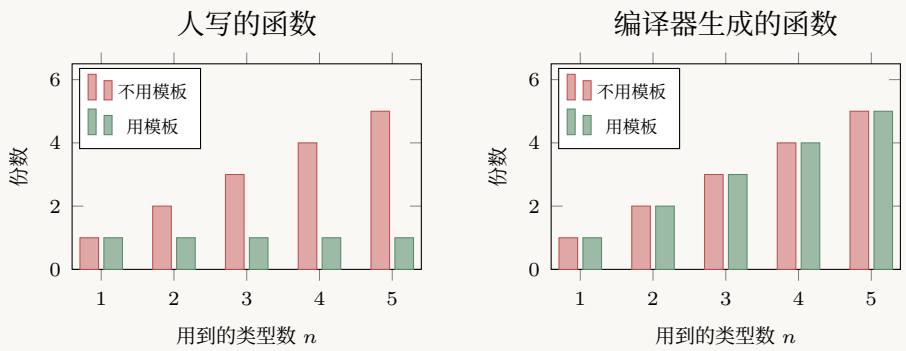


图 6　模板减少的是人写的代码（左），机器生成的函数份数不变（右）

生活中的它 请假条模板

生活场景：班长要为 30 位同学分别写请假条，内容都是“兹有 ___ 同学因 ___ 请假 ___ 天”，只有空格不同。聪明的做法是印一张带空格的模板，每人填自己的。

对应关系：见下图。**小估算**：手写 30 份，每份 40 字，共 1200 字；用模板只写 1 份 40 字 + 每人填 3 个空——“写正文”的工作量缩小为 1/30。函数模板同理：逻辑只写 1 次。**边界**：请假条模板是人在填，函数模板是编译器在填；而且编译器会检查填进去的类型“合不合适”（3.5 节），纸质模板不会。

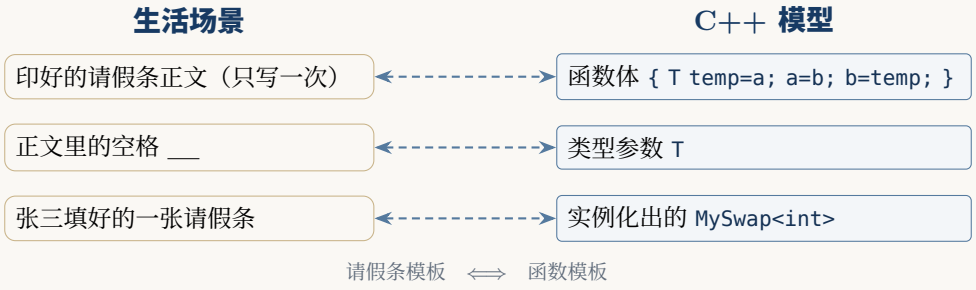


图 7　请假条模板与函数模板的对应

易错 1：以为“重载”就解决了重复

错误想法：“把 `SwapInt`、`SwapDouble` 都改名为 `Swap`，重载一下就行了。”

错在哪：重载只让它们名字相同、调用方便，但函数体仍然要为每种类型写一遍。重载解决“起名字”的问题，模板解决“写代码”的问题。

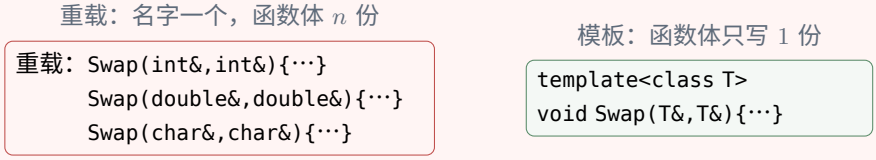


图 8　重载与模板：都能“同名调用”，但只有模板减少了手写的函数体

相近概念辨析：函数重载 vs 函数模板

比较维度	函数重载	函数模板
本质	多个**手写**的同名函数	一个“蓝图”，由编译器**生成**多个函数
不同版本的函数体	可以完全不同	完全相同（只是类型不同）
新增一种类型	必须再手写一个	不用写，直接调用
适用场景	不同类型的处理逻辑不一样	不同类型的处理逻辑一样
常见错误	以为它能减少代码量	以为它对任何类型都能用（见 3.5）

二者可以共存：同名的普通函数和函数模板同时存在时，编译器有一套挑选规则（属于后续内容，本讲义不展开）。

对照：函数模板↔高等代数“数域 P 上的一般理论”（思想重现型）

一句话定位：高等代数在“任意数域 P”上把定理证明一次，就同时对 $\mathbb{Q}$、$\mathbb{R}$、$\mathbb{C}$ 成立；函数模板对“任意合适的类型 `T`”写一次代码，就同时得到 `int`、`double`、`string` 的版本。两者都是“对一整类对象只做一次”。

维度	高等代数：数域 P 上的定理	C++：函数模板
抽象占位	数域 P	类型参数 `T`
“写一次”的是	一个证明	一段代码
具体化	令 $P=\mathbb{R}$，直接得到结论	令 `T=int`，编译器**生成**一份新代码
前提写在哪里	定理条件里白纸黑字（“设 P 是数域”）	C++20 以前不写出来，实例化时才检查
具体化之后	仍是**同一个**定理	是**不同的**函数（地址不同，见 3.3）

相同之处：“抽象一次、具体化多次”的思路相同；具体化后逻辑不变。

不同之处及其根源：数学结论靠逻辑推出，对所有 P 一次性成立，不需要“生成”任何东西；计算机要执行机器指令，而 `int`（4 字节）和 `double`（8 字节）的机器指令不同，所以必须**为每种类型真的生成一份代码**。



图 9　对照图：哪些“数学直觉”可以搬到模板上

迁移判断：第 1 行加条件成立——“合适”指 `T` 必须支持函数体用到的操作（3.5 节）；第 3 行不成立——反例：`nm` 显示 `MySwap<int,double>` 与 `MySwap<double,int>` 是两个地址不同的函数（3.3 节）。

知识联系

后续在 3.3 节你会看到“生成”到底指什么；在第 4 章，同样的思路用到类上，得到类模板；数据结构课里的通用顺序表、栈都建立在这里。

学完这里，你应该能够回答

(1) 两个交换函数的差异只在哪里?
(2) 模板减少的是哪一种“代码”?
(3) 重载和模板各解决什么问题?

3.2　K2　函数模板的语法

学科坐标：C++ 语法　前置：函数定义　后续：类模板语法（4.1）、类外实现（4.3）

知道了“要把类型空出来”，接下来的问题是：**怎么告诉编译器“T 是一个空出来的类型”，而不是一个写错的类名?** 答案就是在函数前加一行模板声明。

核心语法 函数模板

```
template <typename T>
返回类型函数名 (参数列表) { 函数体 }
```

多个类型参数用逗号隔开：`template <typename T, class M>`（源文件写法）。

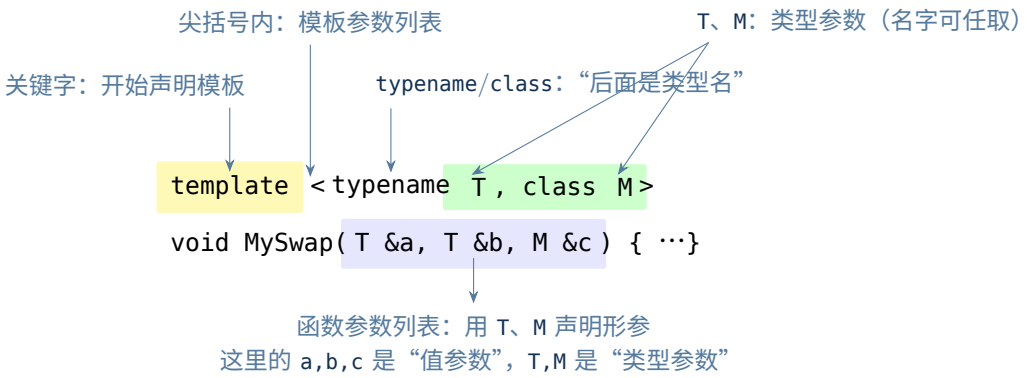


图 10　标注语法图：模板声明的每一部分

图 10 中黄色是关键字，绿色是类型参数，蓝色是函数参数。最需要从图中看出的是：**一个函数模板有两层参数**——尖括号里的“类型参数”（填类型）和圆括号里的“值参数”（填数据）。

记号	含义
`template`	关键字，宣布“接下来的这一个声明是模板”
`< >`	模板参数列表，里面写类型参数
`typename` / `class`	表示紧跟的名字是“一个类型”；**在这里二者完全等价**
`T`、`M`	类型参数名，习惯用大写字母或有意义的名字（如 `NameType`）
`a`、`b`、`c`	普通的函数形参（值参数），类型由 `T`、`M` 决定

规则 模板声明的作用范围（使用条件）

`template <…>` 这一行**只对紧跟在它后面的那一个声明有效**。每写一个函数模板，都要在它前面写自己的 `template <…>`；类型参数名 `T` 出了这个函数就不再有意义。

这条规则正是源文件 类模板分文件.cpp 末尾那个易错点的根源：把函数注释掉却留下了 `template` 行，它就会“接”到下一个声明 `main` 上。

易错 2：孤立的 template 行 【原始资料】

✘ 错误：只注释了函数

```
template <class T1, class T2>
// void
Person<T1,T2>::ShowPerson(){…}
int main() { …}
```

template 行“接”到了 main 上

✔ 正确：连同 template 行一起注释

```
// template <class T1, class T2>
// void
Person<T1,T2>::ShowPerson(){…}
int main() { …}
```

图 11　易错 2：左边 main 被当成模板

实测报错：`error: cannot declare '::main' to be a template`（编译阶段）。

正确想法：`template` 行和它下面的函数是一个整体，要删一起删、要注释一起注释。

易错 3：漏写 typename

写成 `template <T>` 是错的。尖括号里必须先说明“`T` 是一个类型”：`template <typename T>` 或 `template <class T>`。

相近概念辨析：typename vs class　在模板参数列表中二者含义相同，类模板.cpp 注释写 `typename`、代码写 `class` 并没有矛盾。习惯上，有人用 `typename` 强调“任何类型都行（包括 `int`）”，用 `class` 是历史写法。【拓展联想】在模板内部，当需要说明“`T::value_type` 是一个类型”时只能写 `typename T::value_type`，不能写 `class`——这是二者唯一需要区分的地方，以后学 STL 时会遇到。

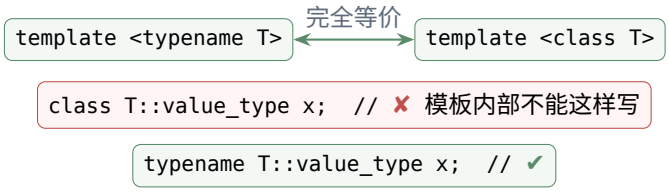


图 12　typename 与 class：参数列表里等价，模板内部有区别（拓展）

例题 2　写一个带两个类型参数的函数模板 【教学补全】

要求：写函数模板 `printPair`，打印两个可能类型不同的值。

思路：两个值类型可能不同，所以需要两个类型参数。

```
template <typename T1, typename T2>   // 两个类型参数
void printPair(T1 x, T2 y)            // x 的类型是 T1，y 的类型是 T2
{
```

```
    cout << "(" << x << ", " << y << ")" << endl;
}
// 调用：printPair(3, 2.5);   推导出 T1=int, T2=double，输出 (3, 2.5)
```

检查：若只用一个 `T`，`printPair(3, 2.5)` 会推导冲突（3.4 节）。**本题想说明**：类型参数的个数取决于“有几处类型可以各自不同”。这也是比原 `MySwap(T&,T&,M&)` 更“语义干净”的双参数示例。

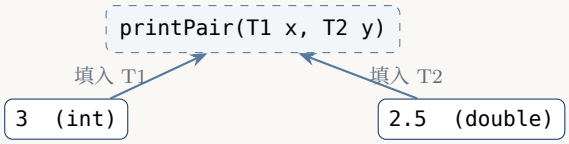


图 13　两个类型参数各自独立地由两个实参确定

生活中的它 “万能菜谱”：清炒 X

生活场景：食堂菜谱写着“清炒 X：热锅凉油，下蒜末，放入 X 大火翻炒 2 分钟，加盐出锅”。X 可以是青菜、土豆丝、豆芽。

对应：见下图。**小判断**：菜谱开头“X 是一种蔬菜”这句话相当于 `typename X`，没有它，读者会以为 X 是一道叫“X”的菜。**边界**：真实菜谱里 X 不同，火候往往要调；而模板生成的代码对每种类型都一字不差，若某种类型需要不同处理，就得另写（以后会学“模板特化”）。

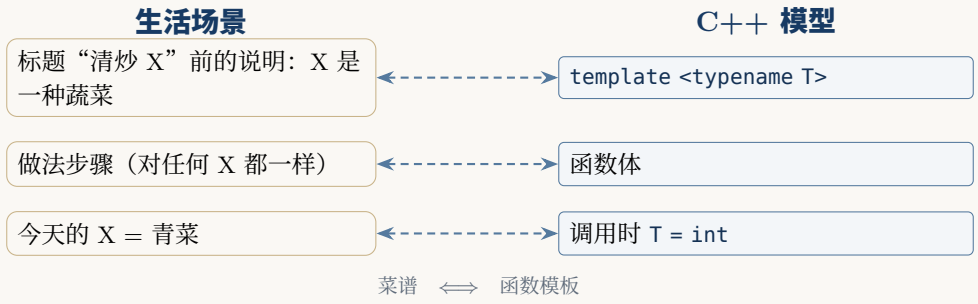


图 14　万能菜谱与函数模板语法的对应

对照：两层参数：数学中的函数族 $f_a(x)$ ↔函数模板 `f<T>(x)`（类比同构型）

一句话定位：数学里 $f_a(x) = ax^2$ 有两层参数：先选定 a 得到一个具体函数，再代入 x 求值；函数模板也是先选定 `T` 得到具体函数，再传入实参执行。

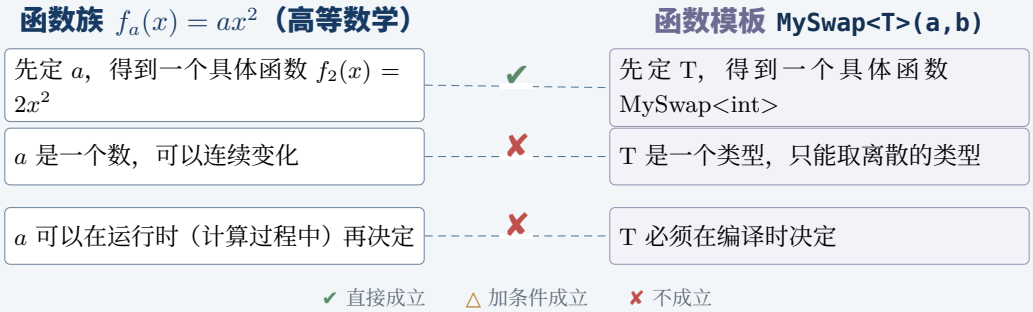


图 15　对照图：两层参数的相同与不同

迁移判断：“先定外层参数、再代入内层参数”这一结构✔ 直接成立；但 `T` 是类型而不是数，必须在**编译时**确定，程序运行时不能“根据用户输入改变 `T`”（✘）。根源：编译器要知道 `T` 才能生成机器指令（3.3 节）。

学完这里，你应该能够回答

(1) 尖括号里的参数和圆括号里的参数有什么不同?
(2) `template` 行管多远?
(3) `typename` 与 `class` 在参数列表中有区别吗?

3.3　K3　模板实例化：从“图纸”到真正的函数

学科坐标：C++ 编译模型　前置：编译与链接　后续：类模板实例化（4.1）、成员函数的实例化时机（4.4）、分文件（第 5 章）

源文件注释里有一句关键的话：【原始资料】“具体什么类型由编译器在调用函数模板时确定。”那么编译器“确定”之后，到底做了什么?

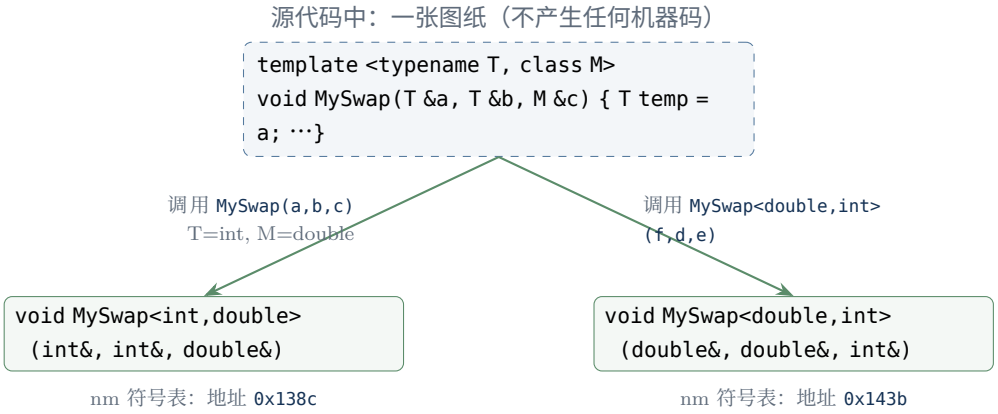


图 16　实例化：源文件的两次调用让编译器生成了两个真实存在、地址不同的函数

图 16 上方虚线框是模板本身，它在源代码里只是“图纸”；下方两个实线框是编译器生成的真实函数。地址取自对源程序可执行文件运行 `nm -C` 得到的符号表：`MySwap<int, double>` 与 `MySwap<double, int>` 是两段不同的机器码。

定义 模板实例化（instantiation）

编译器用具体类型替换模板参数，从函数模板生成一个具体函数的过程，叫做**实例化**；生成的函数叫模板的一个**实例**（又称“特化”，记作 `MySwap<int,double>`）。

隐式实例化：编译器在代码中看到对模板的使用（如调用）时自动进行；**显式实例化**：程序员用 `template void MySwap<int,double>(int&,int&,double&);` 这类语句要求编译器生成（第 5 章用得到）。

规则 实例化的三条基本事实

①实例化发生在**编译期**，不是运行期；　②每一组**不同的**模板实参生成一个实例，同一组实参无论调用多少次只生成一次；　③模板本身不产生机器码，没有被用到的类型组合不会被生成。

推导 / 论证 为什么实例化必须在编译期、并且每种类型要单独生成一份

已知：CPU 执行的是机器指令；`int` 在常见平台占 4 字节、按整数指令运算，`double` 占 8 字节、按浮点指令运算。

第一步：要为 `T temp = a;` 生成机器码，编译器必须知道 `temp` 占多少字节、用什么指令复制——这依赖于 `T` 是什么类型。

第二步：所以在 `T` 未知时无法生成任何机器码，模板只能以“源代码图纸”的形式存在（事实③）。

第三步：`T=int` 与 `T=double` 需要的字节数和指令都不同，因此必须各生成一份（事实②），而这件事只能由编译器在翻译源代码时完成（事实①）。

结论：实例化是编译器在编译期按类型逐一生成代码的过程。□　**一句话**：类型决定机器码，所以“填类型”必须在“出机器码”之前。

推导 / 论证 `MySwap` 确实完成了交换（状态追踪）

设调用前 `a`$=x$、`b`$=y$，逐句追踪三个变量的值：

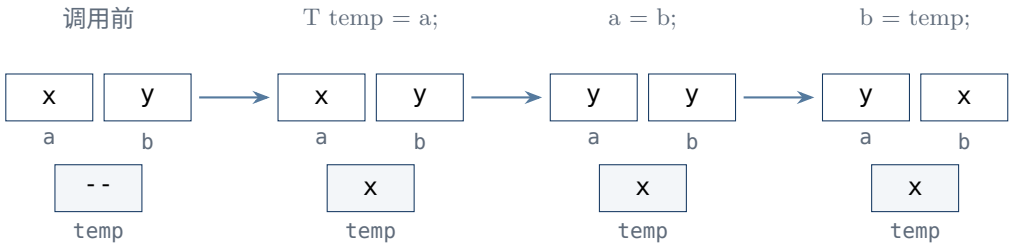


图 17　三步交换的状态追踪（与类型无关）

最后 `a`$=y$、`b`$=x$，交换完成。每一步只用到“复制构造”和“赋值”，没有用到任何类型特有的运算，所以这个论证对所有支持复制和赋值的 `T` 都成立。□

例题 3　下面的程序生成了几个函数实例？

```
template <class T> void mySwap(T &a, T &b) { T t = a; a = b; b = t; }
int main() {
    int x = 1, y = 2;  double p = 1.5, q = 2.5;  int u = 3, v = 4;
    mySwap(x, y);      // ①
    mySwap(p, q);      // ②
    mySwap(u, v);      // ③
}
```

思路：数“不同的类型组合”，不是数调用次数。**第一步**：①推导 `T=int`；② `T=double`；③ `T=int`。**第二步**：不同组合只有 `int`、`double` 两种。**结果**：生成 **2** 个实例，③复用①生成的 `mySwap<int>`。

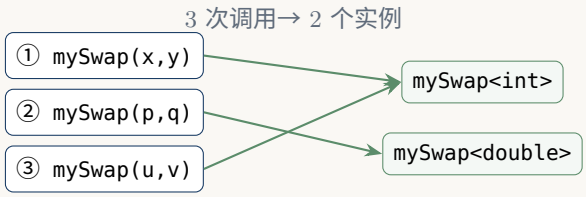


图 18　调用与实例：多对一

生活中的它 月饼模具

生活场景：中秋节用一个木制月饼模具，压出莲蓉、五仁、豆沙三种月饼。模具不能吃；压出来的每一块才是真正的月饼。

小判断：做了 6 块莲蓉、3 块五仁，模具用了 9 次，但“品种”只有 2 种——对应 9 次调用、2 个实例。**边界**：真实的每块月饼是独立的物体；而同一类型的多次调用共用同一个函数实例，不会每调用一次复制一份代码。

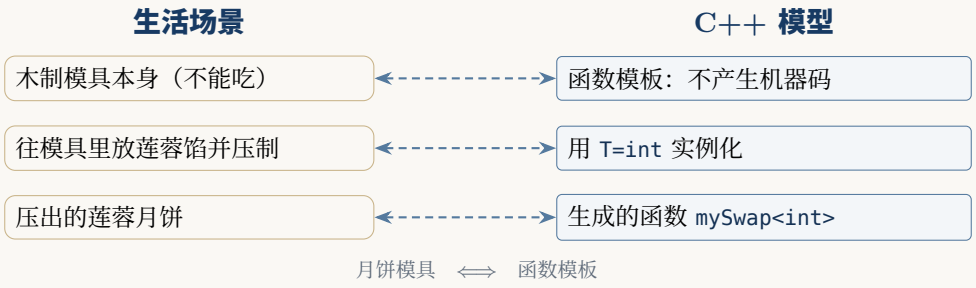


图 19　月饼模具与模板实例化的对应

易错 4：“模板在运行时决定类型”

错误想法：程序运行到调用处，才去看实参是什么类型。**正确想法**：编译时就已决定，运行时已经没有“模板”了，只有生成好的普通函数。

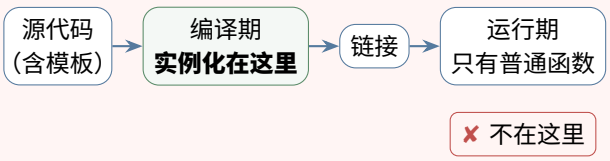


图 20　实例化发生在编译期

相近概念辨析：实例化 vs 调用

	实例化	函数调用
发生时间	编译期	运行期
做了什么	生成函数的机器码	执行已生成的机器码
次数	每种类型组合 1 次	每执行到一次调用语句 1 次
例题 3 中	2 次	3 次

对照：C 语言宏 `#define SWAP` ↔函数模板（方法互补型，跨课程）

一句话定位：二者都是“写一次、对多种类型生成代码”，但宏是预处理器做的**纯文本替换**，模板是编译器做的**带类型检查的代码生成**。

```
#define SWAP(T, a, b) { T t = a; a = b; b = t; }   // C 语言写法
SWAP(int, x, y);   // 预处理后变成 { int t = x; x = y; y = t; }
```



图 21　对照图：宏与模板

不同的根源：宏在编译之前由预处理器处理，它“看不懂”C++，只会替换文字；模板由编译器处理，编译器知道每个表达式的类型。**同一个小算例**：`int x=1; double p=2.7;` 用宏 `SWAP(int,x,p)`，`p` 的值被截断赋给 `int t`，交换后 `x=2`、`p=1`，没有任何错误提示；用模板 `MySwap(x,p)`，编译器直接报“deduced conflicting types”。

学完这里，你应该能够回答

(1) 实例化发生在什么时候?
(2) 调用 5 次 `mySwap<int>` 生成几个实例?
(3) 为什么模板不能等到运行时才决定 `T`?

3.4　K4　两种使用方式：自动类型推导与显式指定类型

学科坐标：C++ 语法与编译期推导　前置：实例化　后续：类模板参数推导（4.1 拓展）、STL 算法调用

实例化需要知道 `T`。`T` 从哪里来? 源文件总结了两种方式：【原始资料】“1. 自动类型推导　2. 显式类型指定”。

核心语法 两种调用方式

自动类型推导：`MySwap(a, b, c);`　编译器根据实参 `a,b,c` 的类型推出 `T`、`M`。

显式指定类型：`MySwap<double, int>(f, d, e);`　尖括号里**按顺序**直接给出 `T=double`、`M=int`。

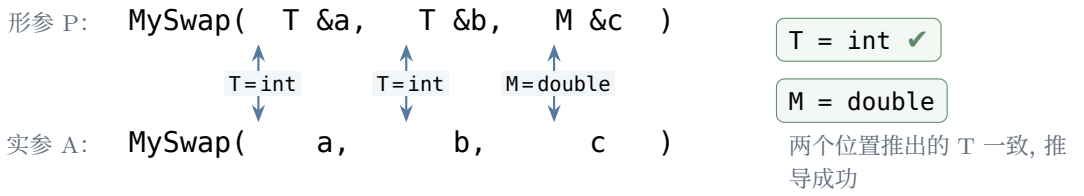


图 22　自动类型推导：逐个位置把形参中的 T、M 与实参类型对齐（a,b 为 int，c 为 double）

图 22 上一行是形参，下一行是实参，竖线表示“对齐比较”。每条竖线给出一个方程，比如第 1 条说“T 必须是 int”。所有方程同时成立，推导才成功。

规则 类型推导的规则（本科阶段够用的版本）

1. 编译器把每个形参类型（如 T&）与对应实参的类型对齐，推出模板参数；
2. 同一个模板参数在各处推出的结果必须**完全一致**，否则推导失败；
3. 推导过程**不做隐式类型转换**（int 不会为了迁就而被当成 double）；
4. 模板参数若没有出现在任何函数形参中，就无法推导，必须显式指定；
5. 显式指定时按顺序填写，未填的再交给推导；显式指定后，按值传递的形参允许普通的隐式转换，引用形参仍要求类型匹配。

推导 / 论证 为什么 MySwap(a, c)（a 为 int，c 为 double）推导失败

考虑 template<class T> void MySwap(T&a, T&b) 与调用 MySwap(a, c)。

第一步：对齐第 1 个形参：T& 对 int，得方程 T = int（规则 1）。

第二步：对齐第 2 个形参：T& 对 double，得方程 T = double。

第三步：两方程要求 T 同时等于两种不同类型，矛盾（规则 2）；而编译器也不会把其中一个实参“转换一下”来消除矛盾（规则 3）。

结论：没有任何 T 满足要求，没有可调用的实例。实测：error: no matching function for call to 'mySwap(int&, double&)'，附注 deduced conflicting types for parameter 'T' ('int' and 'double')。□

条件为什么不能删：如果允许隐式转换，c 会被转成 int，交换时小数部分静默丢失——规则 3 正是为了防止这种悄悄出错。

例题 4　源文件中的两次调用　【原始资料】

已知：int a=10, b=20; double c=1.1; 以及 double f=1.1, d=2.2; int e=1;

第一次：MySwap(a,b,c)：由图 22，T=int、M=double。交换后 a=20, b=10；c=1.1；m = 20+10+1.1 = 31.1。

第二次：MySwap<double,int>(f,d,e)：显式给出 T=double、M=int，与实参类型恰好一致。交换后 f=2.2, d=1.1；c=1.1 赋给 int 被截断为 1；m = 2.2+1.1+1 = 4.3。

检查：实际运行输出 m = 31.1, a = 20, b = 10, m = 4.3, f = 2.2, d = 1.1，一致。

本题想说明：显式指定只是“跳过推导、直接告诉编译器”，生成的实例和推导出来的没有区别。

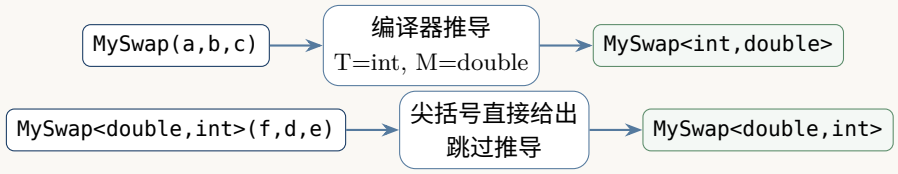


图 23　两条路，终点都是一个具体实例

例题 5　什么时候必须显式指定？【教学补全】

```
template <class T> void func() { cout << "func 调用" << endl; }
func();        // 错误：couldn't deduce template parameter 'T'
func<int>();   // 正确：T 没有出现在形参里，只能显式给出
```

另一种情况：`template<class T> T myAdd(T a, T b)` 中，`myAdd(1, 2.5)` 推导冲突；`myAdd<double>(1, 2.5)` 则可以——显式给出 `T=double` 后，`1` 按值传递，可隐式转换为 `1.0`，结果 `3.5`（规则 5，已实测）。

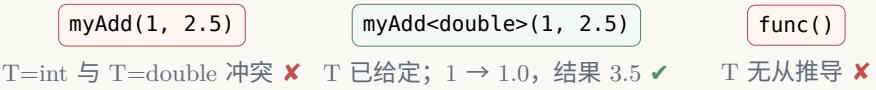


图 24　必须显式指定的两种典型情形

生活中的它 相机的自动对焦与手动对焦

生活场景：手机拍照通常自动对焦——相机根据画面判断焦点；但当画面里有两个距离不同的主体（近处的花、远处的人），自动对焦会“犹豫”，这时你点一下屏幕，手动指定对焦点。

小判断：拍“近花 + 远人”对应哪一种模板调用？——对应 `myAdd(1, 2.5)`：两个实参“指向”两个不同的 `T`，自动推导失败，需要显式指定。**边界**：相机会妥协选一个焦点；编译器不会妥协，推导冲突就直接报错。

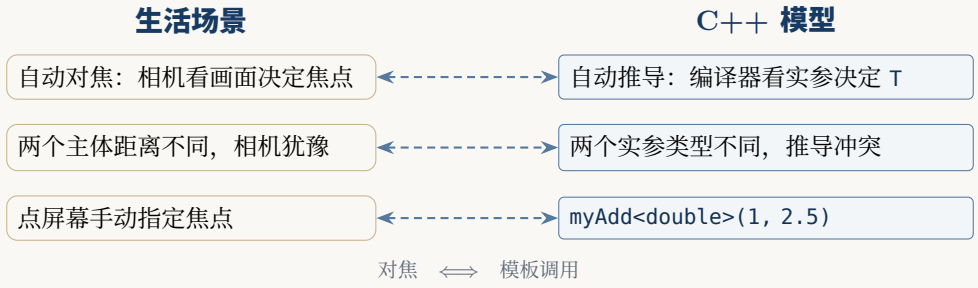


图 25　对焦与类型推导的对应

易错 5：显式指定后，引用形参仍然不能“转换”

`int a=1; double c=2; MySwap<double>(a, c);`　依旧报错：`cannot bind non-const lvalue reference of type 'double&' to a value of type 'int'`。因为 `MySwap` 的形参是 `double&`，它必须“绑定到一个真正的 `double` 变量”，而 `a` 是 `int`。



图 26　引用无法绑定到另一种类型的变量

相近概念辨析：自动推导 vs 显式指定

	自动类型推导	显式指定类型
写法	`MySwap(a,b)`	`MySwap<int>(a,b)`
T 由谁决定	编译器根据实参	程序员写在尖括号里
实参类型不一致时	推导失败	按值形参可隐式转换；引用形参仍失败
T 不在形参中时	无法使用	唯一办法
推荐	类型明确时使用，简洁	需要“说清楚”或推不出来时使用

对照：类型推导↔待定系数法（类比同构型）

一句话定位：待定系数法把 $ax+b\equiv 3x+5$ 两边“同类项对齐”，得到 $a=3$、$b=5$；类型推导把形参 `T&` 与实参 `int` “逐位置对齐”，得到 `T=int`。两者都是“对齐→列方程→求未知”。



图 27　对照图：待定系数法与类型推导

同一个小算例：$\{a=1,\ a=2.5\}$ 无解；`myAdd(1, 2.5)` 推出 `{T=int, T=double}`，同样“无解”。第 4 行是唯一不能迁移的地方：数学里求近似解有时是合理的，但类型推导要求“精确相等”。

学完这里，你应该能够回答

(1) `MySwap(a,c)`（int 与 double）为什么失败？
(2) 什么情况下必须显式指定？
(3) 显式指定后，引用形参能否接受别的类型？

3.5　K5　模板对类型的隐含要求

学科坐标：C++ 泛型编程　前置：实例化　后续：运算符重载、C++20 Concepts、STL 对元素类型的要求　相关：高等代数、复变函数（复数无大小）

到目前为止，似乎“任何类型都能往 T 里填”。可源文件的 MySwap 里有一句 double m = a + b + c;——如果 T 是 string 会怎样?

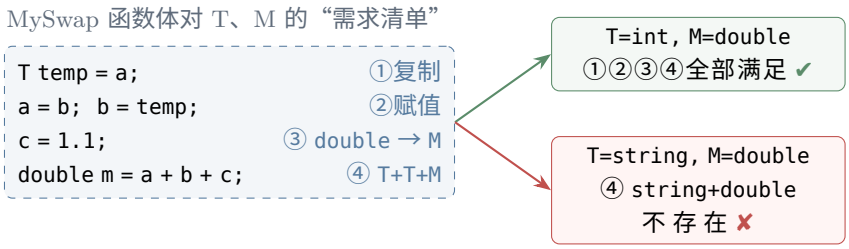


图 28　模板函数体里用到的每个操作，都是对类型的一条要求

图 28 左边把 MySwap 的每一行翻译成一条要求。T=int, M=double 全部满足；换成 T=string 时，第④条 string + double 不存在，实测报错：no match for 'operator+' (operand types are 'std::string' and 'double')。

规则 类型要求规则

函数模板的函数体对模板参数用到的每一个操作（复制、赋值、+、<、<<、调用某个成员函数……）都构成一条**隐含要求**。用某个类型实例化时，只要有一条不满足，**这次实例化**就编译失败；满足全部要求的类型都能用。在 C++20 以前，这些要求不写在模板声明里，只能读函数体才知道。

推导 / 论证 通用 myMax 能用于哪些类型?

```
template<class T> T myMax(T a, T b) { return a < b ? b : a; }
```

需求清单：(i) T 能按值复制（传参、返回）；(ii) 两个 T 能用 < 比较，结果可当作真假。

逐类型检查：int、double：(i)(ii) 都满足 ✔；string：标准库为它定义了按字典序比较的 <，满足 ✔；std::complex<double>：数学上复数没有与运算相容的大小顺序，标准库也没有为它定义 <，(ii) 不满足 ✘，实测 no match for 'operator<'；自定义的 Person：若没有自己写 operator<，(ii) 不满足 ✘。

结论：myMax 恰好对“定义了 < 的可复制类型”可用。□

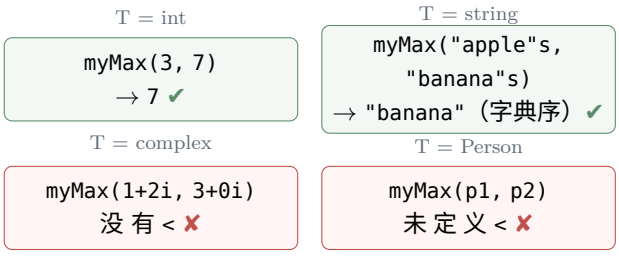


图 29　同一个 myMax，对四种类型的实例化结果（2×2 对照）

易错 6：“满足语法”不等于“结果正确”——源文件中的截断

M=int 时 c = 1.1; 能编译（double 可以隐式转为 int），所以 MySwap<double,int> 实例化成功——但 e 得到的是 1。另一个例子：myMax('a','Z') 返回 'a'，因为字符按编码比较，'Z'（90）小于 'a'（97），未必符合“字母顺序”的直觉。

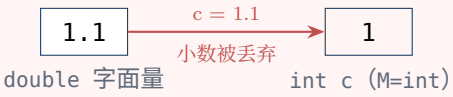


图 30　语法上合法、语义上丢失数据的实例化

正确想法：写模板时想一想“哪些类型会被填进来”，避免在函数体里写只对某些类型有意义的语句。

生活中的它 微波炉的“通用加热”

生活场景：微波炉的“加热 2 分钟”对陶瓷碗、玻璃碗、塑料饭盒（标有可微波）都适用，但金属碗绝对不能放进去。

小判断：“加热”这个操作对容器提出了什么要求？——“能被微波穿透且不打火”。对应 `myMax` 对 `T` 的要求“能用 `<` 比较”。**边界**：金属碗放进去要真的通电才出事（运行时）；模板的不满足在**编译时**就被发现，程序根本生成不出来——这比微波炉安全。

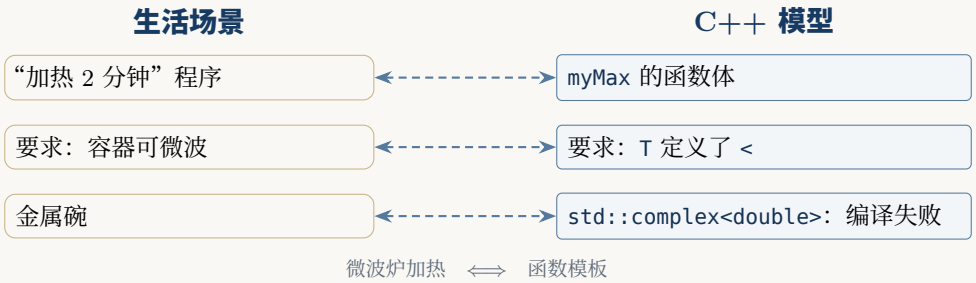


图 31　微波炉与模板类型要求的对应

对照：`myMax<T>` ↔数学中“有序域”上的结论（推广型，跨课程 • 完整版）

一句话定位：在高等代数里，大多数结论对任意数域成立，但凡用到“大小比较”的结论（如“取最大值”“正定”），只对 $\mathbb{Q}$、$\mathbb{R}$ 这样的**有序域**成立，对 $\mathbb{C}$ 不成立；模板的情况完全平行：`myMax` 只对定义了 `<` 的类型可用，对 `std::complex` 编译失败。

维度	数学：数域上的结论	C++：模板的实例化
一般部分	加减乘除、线性方程组、行列式：对任意数域成立	复制、赋值：对几乎所有类型成立
需要额外结构的部分	比大小、取最大值：需要有序域	`myMax`：需要 `operator<`
$\mathbb{C}$ / `complex`	没有与运算相容的大小顺序	标准库不提供 `<`
不满足时	结论不成立（需要另找办法，如比较模长 $\lvert z\rvert$）	编译失败（需要另写比较方式）
条件写在哪	定理的前提里	C++20 前隐含在函数体里；C++20 可用 Concepts 写出

相同之处：都是“一般框架 + 某些结论需要额外结构”；都可以通过“换一个比较标准”（比模长）绕开。

不同之处及其根源：数学里“不成立”需要你自己发现；编译器会**自动检查**每一条要求，不满足就拒绝生成代码。根源是：编译器实例化时必须真的找到 `<` 这个函数

才能生成调用指令。

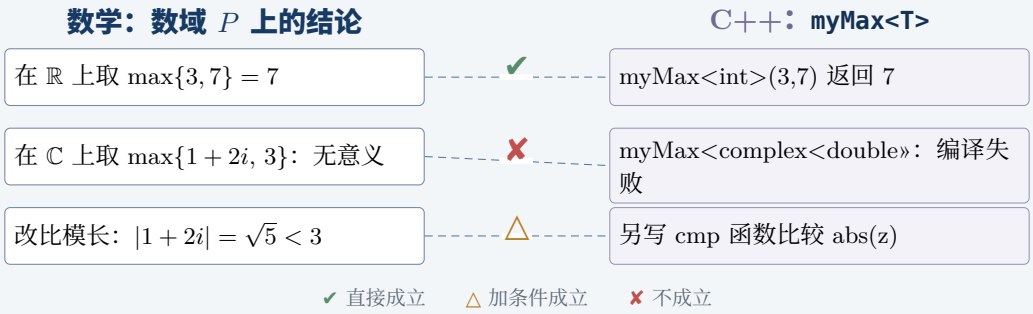


图 32　对照图：“有序”这条额外结构在两边的表现

迁移判断表：

数学中的结论	迁移到模板	理由或反例
对任意数域成立的运算（加、乘）可放心使用	✔	`int`、`double`、`complex` 都定义了 `+`、`*`
在 ℝ 上成立的“比大小”结论	△	只对定义了 `<` 的类型成立
在 ℂ 上可以“取最大值”	✘	反例：`myMax(z1, z2)` 编译失败
改用模长比较可解决	△	需要显式写出比较规则（如传入比较函数）

同一个小算例：数学：$\max\{3,7\} = 7$；$\max\{1+2i, 3\}$ 无定义，但 $|1+2i| = \sqrt{5} \approx 2.24 < |3| = 3$，按模长“3 更大”。C++：`myMax(3,7)` 得 `7`；`myMax(z1,z2)` 编译失败；`abs(z1) < abs(z2)` 为真，结论与数学一致。

教学补全 C++20 Concepts：把隐含要求写出来 【拓展联想】

`template <std::totally_ordered T> T myMax(T a, T b);` 表示“`T` 必须是全序可比较的类型”。这样用 `complex` 调用时，报错会直接说“约束不满足”，而不是一长串函数体内部的错误。当前阶段了解即可。

学完这里，你应该能够回答

(1) 怎样从函数体读出模板对类型的要求?
(2) `myMax` 为什么不能用于复数?
(3) 实例化成功是否意味着结果一定正确?

本章知识地图

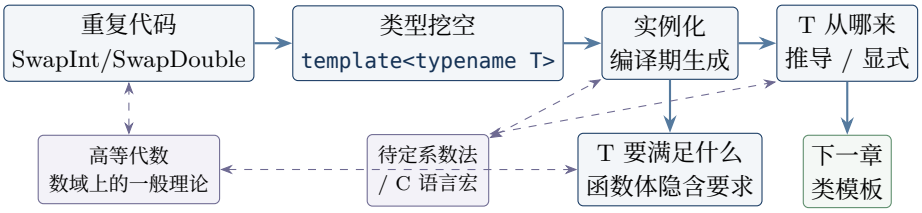


图 33　第 3 章知识地图（虚线：与其他课程中相似知识的对应）

到这里，我们已经解决了什么？下一步呢？

我们已经能写函数模板、知道编译器如何把它变成真正的函数、知道 `T` 从哪里来、也知道 `T` 不能乱填。但数据结构课需要的往往不是一个“通用函数”，而是一个“通用的类”：一个能装 `int` 也能装 `string` 的顺序表。**类能不能也把类型空出来？**这就是下一章的类模板。

第 4 章　类模板

上一章结束时留下了一个问题：数据结构需要的是“元素类型待定的类”。本章把模板从函数推广到类，依次学习：类模板的写法与实例化（4.1）、类模板对象怎样传给函数（4.2）、成员函数写到类外时的语法（4.3），以及一个会在下一章引发“链接错误”的关键事实——成员函数用到才实例化（4.4）。

本章回答三个问题：①类模板和函数模板有什么不同？②类外实现为什么要写那么长？③类模板的成员函数什么时候才真正生成？　**知识路线：**类模板语法 → 模板类与对象 → 作函数参数 → 类外实现 → 用到才实例化。

4.1　K6　类模板的语法与实例化

学科坐标：C++ 类与对象·泛型编程　前置：K2、K3、构造函数、`this`　后续：STL 容器、数据结构中的 `SeqList<T>`、`Stack<T>`　相关课程：数理统计（分布族）

它解决什么问题　在 类模板.cpp 中，`Person` 有姓名和年龄两个成员。如果有时年龄要用 `int`（20 岁），有时要用 `double`（20.5 岁），有时姓名想用 `string`、有时想用编号 `int`，就要写一堆几乎相同的类。和函数一样，类也可以把成员的类型空出来。

核心语法 类模板　【原始资料】

```
template <class NameType, class AgeType>
class Person {
public:  Person(NameType name, AgeType age);  NameType m_Name;
AgeType m_Age;
};
使用：Person<string, int> p(" 张三", 20);
```

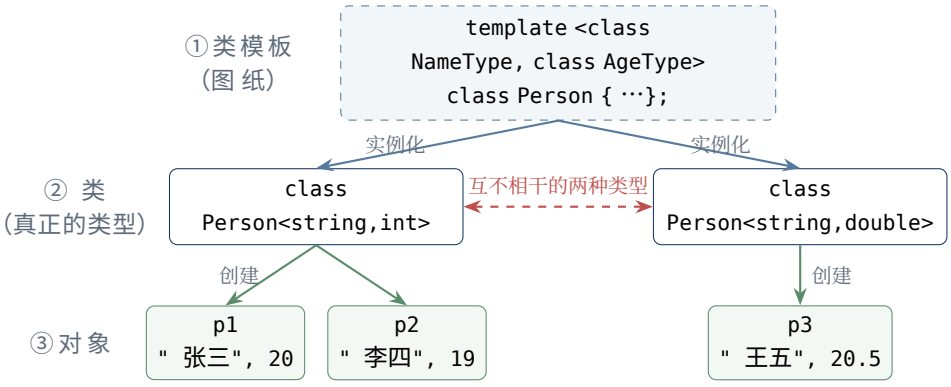


图 34　类模板的三层结构：类模板→类（模板类）→对象

图 34 要从上往下读：最上层是图纸，第二层是编译器生成的两个具体类，第三层才是程序运行时创建的对象。第二层的两个类之间的红色虚线提醒你：它们虽然出自同一张图纸，却是两种互不相干的类型。

> **一句话直观理解**　类模板是“类的图纸”，`Person<string,int>` 是按图纸造出的一种“类”，`p` 是这种类的一个“对象”。函数模板有一次“生成”，类模板到对象有两次。

定义 类模板

类模板（class template）：带有类型参数的类的“蓝图”。给出全部模板实参后，如 `Person<string,int>`，得到的是一个真正的类，常称为“模板类”或类模板的一个特化。

注意：`Person` 单独出现时不是一个类型；`Person<string,int>` 才是。

规则 类模板的使用规则

1. 声明对象、写函数参数、写返回类型时，要用完整的 `Person< 实参列表 >`；
2. 模板实参不同，得到的就是不同的类型，彼此不能直接赋值；
3. 【教学补全】类模板的参数可以有默认值：`template <class NameType, class AgeType = int>`，之后 `Person<string>` 等价于 `Person<string,int>`（函数模板的默认参数规则不同，本讲义不展开）；
4. 【教学补全】在 C++17 之前，创建类模板对象**不能**省略尖括号；C++17 起可由构造函数实参推导（见下方易错 7）。

推导 / 论证 为什么 `Person<int,int>` 与 `Person<int,double>` 必须是两种类型

第一步：一个类型首先决定对象的**内存布局**。`Person<int,int>` 的对象含两个 `int`，实测 `sizeof` 为 8 字节；`Person<int,double>` 含一个 `int` 和一个 `double`，加上对齐填充，实测为 16 字节。

第二步：大小不同的对象不可能用同一套机器指令来复制、访问成员；`m_Age / 2` 在一个里是整数除法，在另一个里是浮点除法。

第三步：因此编译器必须把它们当作两个独立的类，各自生成成员函数；它们之间也没有自动定义的赋值运算。

结论：实测 `a = b;`（`a` 为 `Person<string,int>`，`b` 为 `Person<string,double>`）报错 `no match for 'operator='`。□

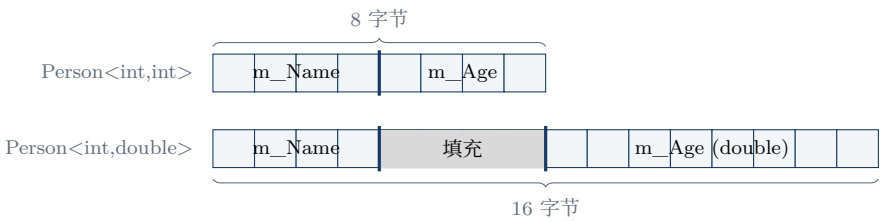


图 35　两种模板类的对象内存布局（每格 1 字节，64 位 g++ 实测）

例题 6　源文件中的类模板　【原始资料】

```cpp
template <class NameType, class AgeType>
class Person {
public:
    Person(NameType name, AgeType age) { this->m_Name = name; this->m_Age =
    age; }
    NameType m_Name;
    AgeType  m_Age;
};
Person<string, int> p("张三", 20);   // NameType=string, AgeType=int
```

第一步：尖括号按顺序把 `string` 填给 `NameType`，`int` 填给 `AgeType`，编译器生成类 `Person<string,int>`。**第二步**：调用它的构造函数，`" 张三"` 转换为 `string` 存入 `m_Name`，`20` 存入 `m_Age`。**结果**：输出“姓名：张三年龄：20”。

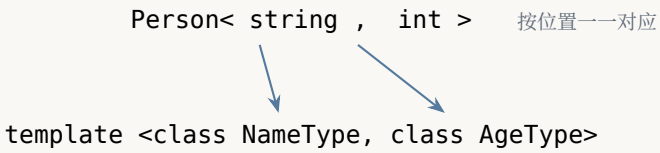


图 36　类模板实参按位置对应模板参数

易错 7：`Person p(" 张三", 20);` 在不同标准下结果不同

C++14 及以前：报错 `missing template arguments before 'p'`。C++17 起：可以编译（类模板参数推导），但推出的是 `Person<const char*, int>`——姓名成了 C 风格字符串指针，而不是 `string`（已实测）。所以**初学阶段请总是写全 `Person<string,int>`**。

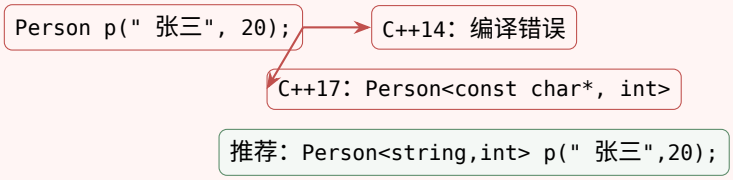


图 37　省略尖括号的两种后果

易错 8：两个“实例化”不是一回事

“用类模板生成类”和“用类创建对象”都常被叫做“实例化”。前者在编译期、产物是类型；后者在运行期、产物是对象（见本节开头的三层结构图）。说话时最好说清楚：“模板实例化”还是“创建对象”。

相近概念辨析：函数模板 vs 类模板

	函数模板	类模板
生成物	函数	类（再由类创建对象）
使用时尖括号	通常可省略（自动推导）	C++17 前必须写全；初学建议总写全
默认模板参数	可以有，但规则不同	常用：`class AgeType = int`
成员何时生成	整个函数一起生成	每个成员函数**用到才**生成（4.4）
典型例子	`MySwap`、`sort`	`Person`、`vector`、`stack`

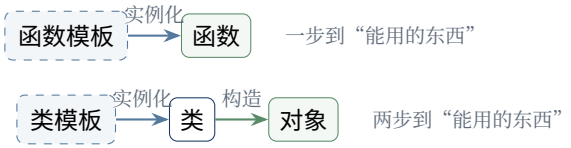


图 38　函数模板一步、类模板两步

生活中的它 分格便当盒

生活场景：一种便当盒的盒子形状固定（两格），左格放主食、右格放配菜。今天装“米饭 + 青椒肉丝”，明天装“面条 + 卤蛋”。

小判断：一个“米饭格”便当盒和一个“面条格”便当盒能互换内容吗？不能直接倒——对应 `Person<string,int>` 与 `Person<string,double>` 不能直接赋值。**边界**：现实中盒子是同一种盒子，只是装的不同；而模板生成的 `Person<string,int>` 与 `Person<string,double>` 连“盒子”都是两种（内存布局不同）。

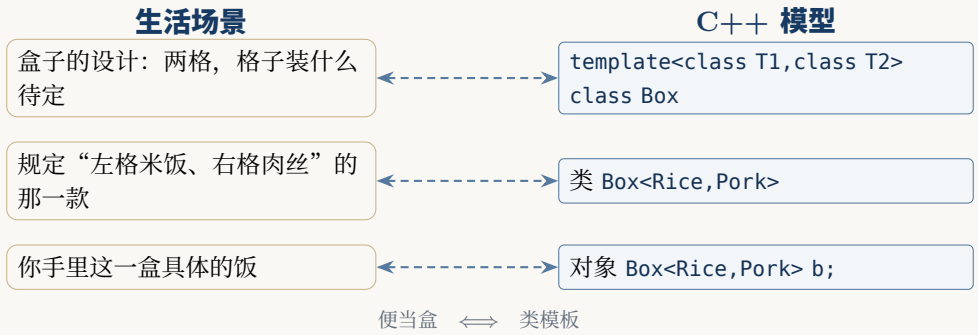


图 39　便当盒与类模板三层结构的对应

对照：类模板↔数理统计中的“分布族”（类比同构型，跨课程）

一句话定位：数理统计里，正态分布族 $N(\mu,\sigma^2)$ 是“带参数的一整族分布”，给定 $\mu=0,\sigma^2=1$ 就得到一个具体分布 $N(0,1)$,再从中抽出样本值;类模板 `Person<T1,T2>` 给定类型得到具体类，再创建对象——同样是三层。



图 40　对照图：分布族与类模板

不同之处的根源：统计里的参数是数，所以能在样本里“估计”它；模板参数是类型，必须由程序员或编译器在编译时给定。第 4 行加条件：公式对整个族成立是数学证明；成员函数对整个模板“写一次”，但只有用到的类型、用到的成员才会被生成，并且要满足 3.5 节的类型要求。

学完这里，你应该能够回答

(1) `Person` 与 `Person<string,int>` 哪个是类型?
(2) 为什么两种模板类不能互相赋值?
(3) C++17 中省略尖括号会推出什么?

4.2　K7　类模板对象作函数参数

学科坐标：C++ 函数与类模板的结合　前置：K4（推导）、K6　后续：写通用的打印、比较、遍历函数

类模板.cpp 里写了一个打印函数：【原始资料】`void Showdetail(Person<string, int> &p)`。这引出一个问题：**函数的形参能写成“某个类模板的对象”吗?** 能，但有三种写法，覆盖范围不同。

核心语法 三种传入方式

①指定传入类型 【原始资料】　`void Showdetail(Person<string,int> &p);`
②参数模板化 【教学补全】　`template<class T1,class T2> void Show2(Person<T1,T2> &p);`
③整个类模板化 【教学补全】　`template<class T> void Show3(T &p);`

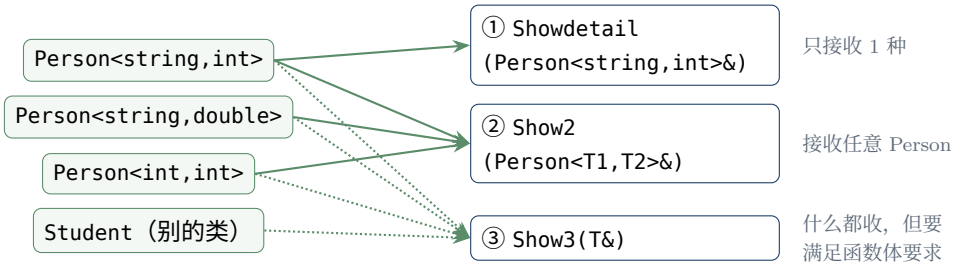


图 41　三种写法各能接收哪些实参：从左边的实参看箭头指向哪些函数

图 41 中，越往下的写法接收范围越大：①只接收 `Person<string,int>`；②接收任何

Person<…>；③连 Student 也能传进来——前提是函数体里用到的 p.m_Name 等成员 Student 也有（3.5 节的类型要求）。

推导 / 论证 写法②是怎样推导出 T1、T2 的

调用 Show2(p)，p 的类型为 Person<string,int>。

第一步：把形参类型 Person<T1,T2> 与实参类型 Person<string,int> 对齐（K4 规则 1）。**第二步**：两边模板名都是 Person，于是尖括号里逐位比较：T1 = string，T2 = int。**第三步**：没有冲突（规则 2），推导成功，生成 Show2<string,int>。□

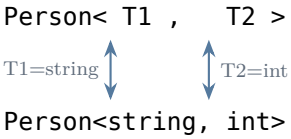


图 42　写法②的推导：模板名对模板名，尖括号逐位对齐

例题 7　哪些调用能通过编译?

已知三个函数如上，Person<string,double> q(" 王五", 20.5);。分别调用 Showdetail(q)、Show2(q)、Show3(q)。

分析：Showdetail 只收 Person<string,int>，而 q 是另一种类型（K6 规则 2），**不能**；Show2 推出 T1=string, T2=double，**能**；Show3 推出 T=Person<string,double>，函数体里的 p.m_Name、p.m_Age 都存在，**能**。**本题想说明**：参数写得越“具体”，能接收的实参越少；写法①最常用、可读性最好。

易错 9：形参里只写 Person

void show(Person &p);　报错　missing template argument list after 'Person'。原因：Person 本身不是类型（K6）。要么写全 Person<string,int>，要么把函数也变成模板（写法②）。

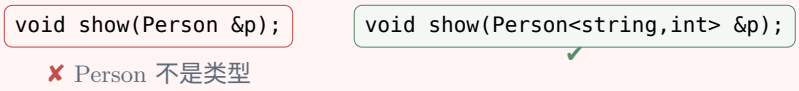


图 43　形参必须是完整的类型

生活中的它 充电接口

生活场景：老式手机只能插原装充电线（只认一种）；Type-C 口能插各品牌的 Type-C 线（认一类）；多口转接头什么线都能插，但前提是线另一头真的能供电。

小判断：一根 Lightning 线能插进 Type-C 口吗？不能——对应 Person<int,int> 对象传给写法①。**边界**：插错口在物理上插不进去；类型不匹配在编译时就被拒绝，同样“插不进去”。

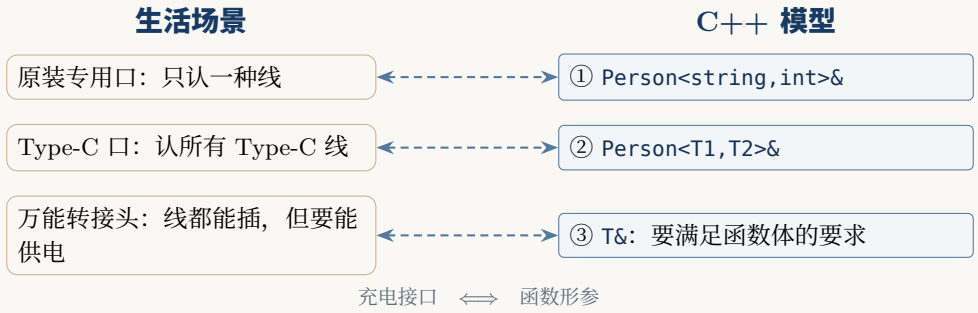


图 44　充电接口与三种传参写法的对应

对照：函数形参的类型↔数学中函数的定义域（类比同构型）

一句话定位：数学函数 $f : D \to \mathbb{R}$ 只能在定义域 D 内取值；C++ 函数只能接收形参类型允许的实参。写法①②③相当于把“定义域”从一个点扩大到一族，再扩大到“任何满足条件的对象”。

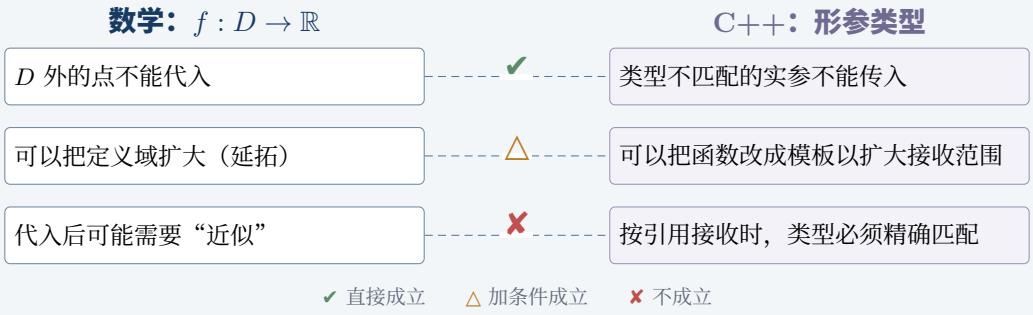


图 45　对照图：定义域与形参类型

迁移判断：第 2 行“加条件”：数学延拓要保证公式在新点仍有意义；模板化要保证新类型满足函数体的要求（3.5 节）。

学完这里，你应该能够回答

(1) 三种写法各接收什么？
(2) 写法②怎样推出 `T1`、`T2`？
(3) 为什么 `show(Person &p)` 不对？

4.3　K8　类模板成员函数的类外实现

学科坐标：C++ 类的组织　前置：普通类成员函数的类外实现（**类名:: 函数名**）、K2 中“`template` 行只管下一个声明”　后续：分文件编写（第 5 章）

类的成员函数一多，把函数体都写在类里会让类变得很长。上一个专题“封装”中你已经会把普通类的成员函数写到类外。`类模板成员函数类外实现.cpp` 把同样的做法用到了类模板上。

核心语法 类模板成员函数的类外实现 【原始资料】

```
template <class T1, class T2>
返回类型 Person<T1, T2>:: 函数名 (参数列表) { …}
```

构造函数没有返回类型：

```
template <class T1, class T2>
Person<T1,T2>::Person(T1 name, T2 age) { …}
```

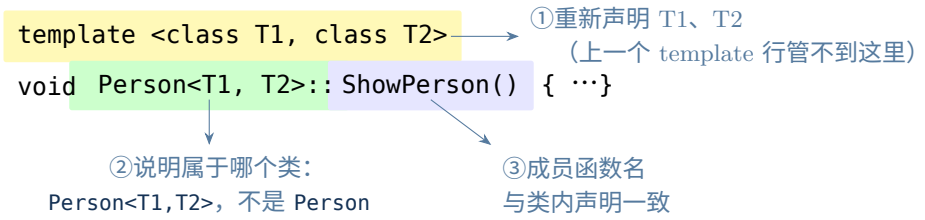


图 46　标注语法图：类外实现的三个组成部分

图 46 中三种颜色分别回答三个问题：① T1、T2 是什么？——要重新声明；②这是谁的成员？——Person<T1,T2> 的；③是哪个成员？——ShowPerson。

推导 / 论证 从普通类的类外实现“推导”出类模板的写法

起点：普通类 class Person 的类外实现是 void Person::ShowPerson() { …}。

第一步：函数体里要用 T1、T2，但类模板开头的 template <class T1, class T2> 只对紧跟着的那个类声明有效（K2 规则），到了类外它们已经“失效”，所以必须在函数前**再写一遍** template <class T1, class T2>。

第二步：类外实现要写“类名::”，而 Person 单独不是类（K6），属于它的类是 Person<T1,T2>，所以写成 Person<T1,T2>::。

第三步：合起来就是

```
template <class T1, class T2>
void Person<T1,T2>::ShowPerson() { …}
```

□

一句话：比普通类多出的两处，分别来自“template 只管一个声明”和“Person 不是类型”这两条已学规则，没有任何需要死记的新东西。

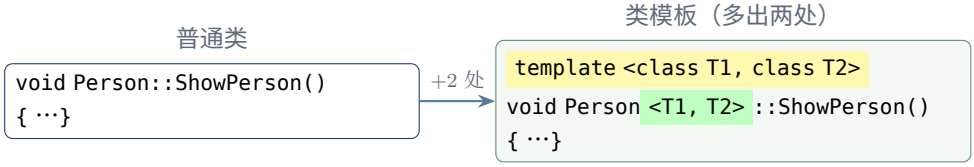


图 47　普通类与类模板的类外实现并排对照

例题 8　补写一个类外实现的成员函数 【教学补全】

要求：给 Person 增加成员函数 void setAge(T2 age);，并在类外实现。

```
1 template <class T1, class T2>
```

```cpp
class Person {
public:
    Person(T1 name, T2 age);
    void ShowPerson();
    void setAge(T2 age);          // 类内只写声明
    T1 m_Name;
    T2 m_Age;
};
template <class T1, class T2>     // ① 重新声明模板参数
void Person<T1, T2>::setAge(T2 age)  // ② Person<T1,T2>::    ③ 函数名与参数
{
    this->m_Age = age;
}
```

检查：形参类型写 `T2`，与类内声明一致；返回类型 `void` 写在 `template` 行之后、`Person<T1,T2>::` 之前。

易错 10：类外实现的三种常见写法



图 48　三种错误写法与正确写法

生活中的它 产品说明书：目录与正文

生活场景：一本“通用型号 X”说明书，前面目录列出“1. 开机　2. 设置”（只有标题），正文在后面。正文每一节的页眉都要写“适用型号：X 系列”，否则读者翻到中间一页，不知道这页讲的是哪个型号。

小判断：类外实现中相当于“页眉”的是哪一部分？——`template <class T1,class T2>` 与 `Person<T1,T2>::`。**边界**：说明书页眉漏写，读者还能猜；编译器不会猜，漏写就报错。

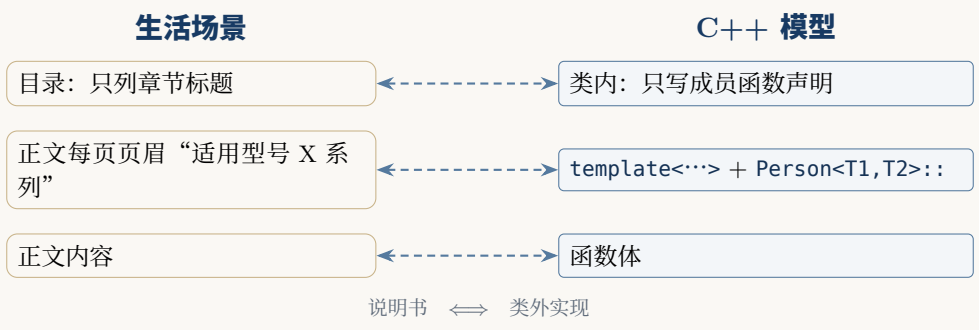


图 49　说明书与类外实现的对应

对照：普通类的类外实现↔类模板的类外实现（推广型，跨专题：“封装”）

一句话定位：类模板的类外实现就是普通类的类外实现再加两样东西：模板参数声明、带尖括号的类名。

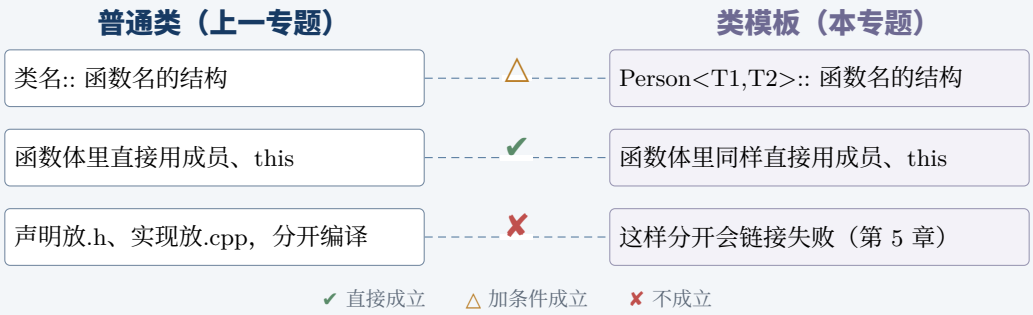


图 50　对照图：两种类外实现

迁移判断：第 1 行“加条件”——条件就是加上 `template<…>` 和 `<T1,T2>`；第 3 行“不成立”是下一章的主题，也是本专题最容易踩的坑。

学完这里，你应该能够回答

(1) 为什么类外每个函数都要重写 `template` 行?
(2) `Person::` 为什么不行?
(3) 构造函数类外实现怎么写?

4.4　K9　成员函数“用到才实例化”

学科坐标：C++ 编译模型　前置：K3、K5、K6　后续：分文件编写（第 5 章）；STL 容器对元素类型的“按需要求”

源文件 类模板分文件.cpp 的第一句话是：【原始资料】“类模板中成员函数创建时机是在调用阶段”。这句话说得更精确一些（见原图纠错 6），就是下面的规则。

规则 类模板成员函数的实例化时机　【教学补全】

用类模板生成一个类（如 `Person<string,int>`）时，编译器只生成类的“外壳”（有哪些成员变量、有哪些成员函数的声明）；**每个成员函数只有在程序中被用到时，才在编译期被实例化**。没有用到的成员函数不会被生成，也不要求它对这个类型“说得通”。

依据：cppreference 对类模板的说明：“unless the member is used in the program, it is not instantiated, and does not require a definition.”

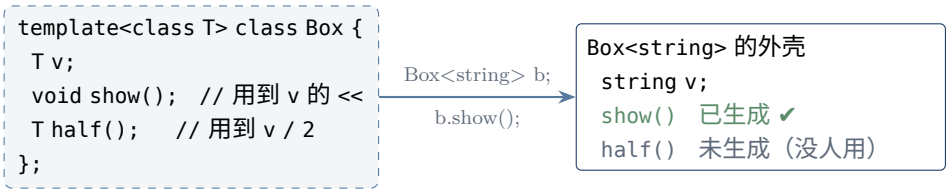


图 51　只调用了 show()，所以只有 show() 被生成；half() 对 string 本来不成立，却不会报错

图 51 中 `half()` 要计算 `v / 2`，对 `string` 显然没有意义（`string` 没有除法）。但只要程

序没调用 `b.half()`，编译就能通过、运行输出 `hi`；一旦加上 `b.half();`，立即报错 `no match for 'operator/'`（均已实测）。

推导 / 论证 为什么这条规则是合理的（也是必要的）

第一步：K5 告诉我们，每个成员函数都对 `T` 提出自己的要求；一个类模板的不同成员函数，要求往往各不相同（`show` 要求 `<<`，`half` 要求 `/`）。

第二步：如果生成类时就把所有成员函数全部实例化，那么 `Box<string>` 会因为 `half` 不成立而整体失败，哪怕你只想用 `show`。

第三步：按需实例化后，一个类型只需满足“它实际用到的那些成员”的要求。STL 正是依赖这一点：例如 `vector<T>` 的某些成员要求 `T` 可默认构造，但你不调用那些成员，`T` 就不必满足。

结论：按需实例化让类模板“对类型的要求”降到最低，同时省去不必要的代码生成。□

易错 11：并不是所有错误都能“躲过去”

只有**依赖于 `T`** 的错误（如 `v / 2`）才会推迟到实例化时检查。与 `T` 无关的错误，比如在模板里调用一个根本不存在的函数 `undeclared_fn();`，g++ 在读到模板定义时就会报错：`there are no arguments to 'undeclared_fn' that depend on a template parameter, so a declaration …must be available`（已实测）。

```
T half() { return v / 2; }
// 依赖 T：用到才检查
```

```
void bad() { undeclared_fn(); }
// 不依赖 T：立即检查
```

图 52　两类错误的检查时机不同

例题 9　判断能否编译　【教学补全】

`Box<string> b("hi");` 之后分别只写① `b.show();` ② `b.half();` ③什么都不调用。

分析：①只实例化 `show`，`string` 支持 `<<`，能；②实例化 `half`，`string / int` 不存在，不能；③只生成外壳和构造函数，能。**本题想说明**：同一个类型、同一个类模板，能否编译取决于“用了哪些成员”。

生活中的它 餐厅菜单：点了才做

生活场景：一家餐厅菜单上有 40 道菜，但后厨只做被点到的菜。今天没有鲈鱼了，只要没人点“清蒸鲈鱼”，餐厅照常营业；有人点了，才发现做不了。

小估算：一桌点 5 道菜，后厨只做 5 道，而不是 40 道——编译器也只生成被调用的成员函数。**边界**：“菜名写错了”（与食材无关的错误）在印菜单时就会被发现——对应易错 11 中不依赖 `T` 的错误。

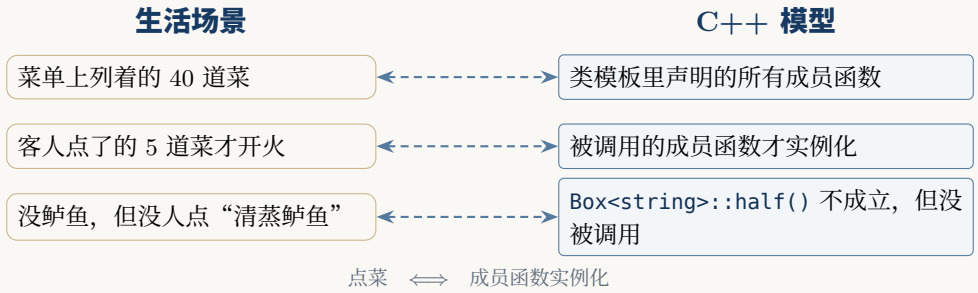


图 53　点菜与“用到才实例化”的对应

对照：模板“用到才检查”↔ Python“运行到才报错”（同名异义型，跨课程）

一句话定位：两者都有“写错了但暂时不报错”的现象，但检查发生在完全不同的阶段：模板在**编译期**、实例化时检查；Python 在**运行期**、执行到那一行时检查。

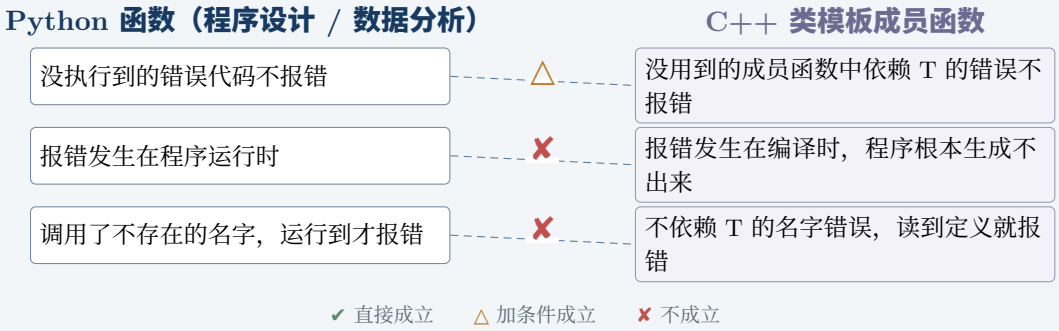


图 54　对照图：两种“延迟检查”

不同的根源：Python 没有编译期类型检查，类型在运行时才确定；C++ 的类型在编译时就必须确定，只是模板要等 `T` 被给出后才能检查和 `T` 有关的部分。

学完这里，你应该能够回答

(1) 生成 `Box<string>` 时，哪些成员函数被生成?
(2) 为什么按需实例化是必要的?
(3) 什么样的错误不会被推迟?

本章知识地图

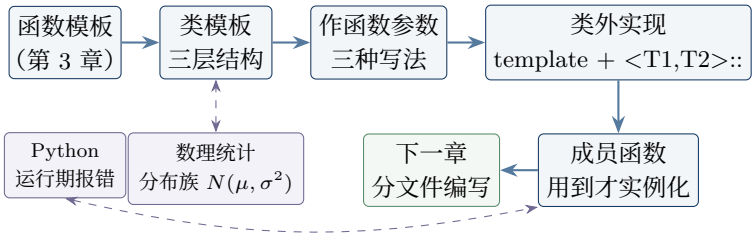


图 55　第 4 章知识地图（虚线：与其他课程中相似知识的对应）

到这里，我们已经解决了什么？下一步呢？

我们会写类模板、会把它的对象传给函数、会把成员函数写到类外，也知道成员函数用到才生成。下一章把这些放进一个真实的工程场景：把类模板拆成 `.h` 和 `.cpp` 两个文件——你会看到，“用到才生成”这条规则恰恰导致了源文件中记录的那个 `undefined reference`。

第 5 章　类模板的分文件编写

上一章最后得到一条规则：类模板的成员函数在编译期、用到时才实例化。本章看它在多文件工程里引起的后果——这正是你在 类模板分文件编写 文件夹里详细记录的问题。**本章回答三个问题：**①为什么“声明放 `.h`、实现放 `.cpp`”对类模板失效？②有哪几种解决办法，各有什么代价？③学会之后，怎样写出数据结构课里的第一个通用容器？　**知识路线：**编译单元 → 实例化需要看到定义 → 链接错误 → 三种解决 → `MyStack<T>`。

5.1　K10　为什么分文件会出现链接错误

学科坐标：C++ 多文件程序·编译模型　前置：编译与链接（1.3 节补全框）、K3、K9　后续：头文件组织、库的设计、显式实例化

问题再现　【原始资料】　按照普通类的习惯，把 `Person` 拆成三个文件：`person.h` 只有类模板声明，`person.cpp` 只有成员函数的类外实现，`main.cpp` 包含 `person.h` 并使用 `Person<string,int>`。执行 `g++ main.cpp person.cpp -o main`，链接时报错（g++ 13 实测，类型名已简写）：

```
undefined reference to `Person<std::string, int>::Person(std::string, int)'
undefined reference to `Person<std::string, int>::ShowPerson()'
```

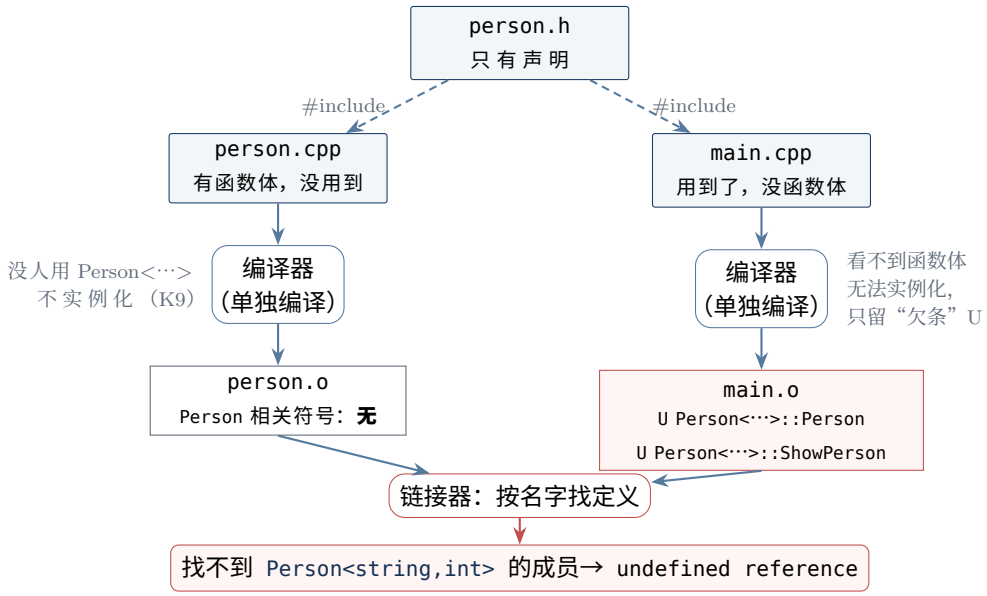


图 56　分文件编译的全过程：两个编译单元各缺一半，链接器两边都找不到

图 56 从上往下读。`person.o` 与 `main.o` 的内容都来自对实际目标文件运行 `nm -C` 的结

果：person.o 中没有任何 Person 成员函数；main.o 中这两个函数被标为 U（undefined，需要别人提供）。链接器拿着 main.o 的两张“欠条”去 person.o 找，找不到，于是报错。

定义 编译单元、外部符号与链接错误

编译单元：一个 .cpp 文件连同它 #include 进来的全部内容。编译器每次只处理一个编译单元，看不到其他 .cpp。

外部符号：在本编译单元中使用、但定义需要由其他目标文件提供的函数或变量（nm 中标为 U）。

链接错误 undefined reference：链接时，某个外部符号在所有目标文件中都找不到定义。

推导 / 论证 链接错误的成因（完整论证）

要说明什么：三文件写法必然导致 Person<string,int> 的成员函数没有定义。

已知：(a) 编译器一次只看一个编译单元；(b) 模板只有在“看到定义 + 被用到”时才实例化（K3、K9）；(c) 链接器只能找到某个目标文件里已有的定义，自己不会生成代码。

第一步（编译 person.cpp）：这里看得到成员函数的定义，但没有任何代码使用 Person<string,int>；由 (b)，不实例化，person.o 中没有这些函数。

第二步（编译 main.cpp）：这里使用了 Person<string,int> 的构造函数和 ShowPerson，但 person.h 中只有声明、没有定义；由 (b)，无法实例化。编译器像对待普通函数声明一样，只记下“需要外部符号”。

第三步（链接）：链接器在两个目标文件中都找不到这两个符号的定义，由 (c) 也无法临时生成，报 undefined reference。

结论：“有定义的地方没人用，有人用的地方没定义”。□　**一句话**：模板的实例化需要**在同一个编译单元里同时看到“定义”和“使用”**。

教学补全 为什么普通类分文件就没事

普通类 Person 的成员函数不是模板，编译 person.cpp 时**无条件**生成它们的机器码，person.o 中就有定义（nm 标为 T）。链接器拿 main.o 的欠条去找，自然找得到。区别的根源只有一个：**普通函数“写了就生成”，模板“用到才生成”**。

例题 10　为什么你的 类模板分文件.cpp 能正常运行 【原始资料】

你的工程里 类模板分文件.cpp 包含的是只有声明的 person.h，却没有报错。

分析：该文件把两个成员函数的类外实现直接写在本文件、并位于 main 之前。于是在这个编译单元里，编译器既看得到定义、又看得到 main 中的使用，可以当场实例化——相当于手动做了“方法 2”。**结论**：问题从来不是“文件后缀”，而是“定义和使用是否在同一个编译单元”。

一个编译单元里：声明 + 定义 + 使用 ✔

```
#include "person.h"                                    ←声明
template<…> Person<T1,T2>::Person(T1, T2) {…}          ←定义
template<…> void Person<T1,T2>::ShowPerson() {…}       ←定义
int main() { Person<string,int> p(…); p.ShowPerson(); } ←使用
```

图 57　你的文件之所以能运行：三样东西都在同一个编译单元里

5.2　三种解决方法

既然根源是“定义与使用不在同一个编译单元”，解决思路就有两条：要么**让使用者看得到定义**（方法 1、2），要么**在有定义的地方主动生成**（方法 3）。



图 58　三种解决方法：前两种让 main.cpp 看到定义，第三种在 person.cpp 中主动生成

核心语法 三种写法

方法 1 【原始资料】　`main.cpp` 中写 `#include "person.cpp"`，编译时只编译 `main.cpp`。

方法 2 【原始资料】　把声明和类外实现写进同一个 `person.hpp`，`main.cpp` 写 `#include "person.hpp"`。`.hpp` 只是约定后缀，编译器并不强制。

方法 3 【教学补全】　保留 `person.h` + `person.cpp`，在 `person.cpp` 末尾加一行**显式实例化**：`template class Person<std::string, int>;`

推导 / 论证 方法 3 为什么有效

显式实例化定义 `template class Person<std::string,int>;` 要求编译器在**当前编译单元**（`person.cpp`，这里看得到定义）生成 `Person<string,int>` 的全部成员。于是 `person.o` 中有了这两个函数的定义，`main.o` 的欠条在链接时就能兑现。实测：加上这一行后 `g++ main.cpp person.cpp` 链接成功，输出 `name: Tom age: 20`。

代价：只有列出的类型可用。若 `main.cpp` 又用了 `Person<string,double>`，仍会 `undefined reference`，除非再加一行显式实例化。

	方法 1：包含.cpp	方法 2：.hpp	方法 3：显式实例化
支持的类型	任意	任意	只限列出的
实现是否对使用者可见	可见	可见	不可见（可隐藏实现）
编译时间	每个包含者都重新编译	同左	只编译一次
主要风险	违反“只包含头文件”的惯例	头文件变长	漏列类型就链接失败
推荐程度	不推荐	**推荐（默认选择）**	类型固定的库中使用

易错 12：以为 `#pragma once` 或“把 person.cpp 一起编译”能解决

`#pragma once` 只防止同一个头文件在一个编译单元里被重复包含，和“看不看得到定义”无关；把 `person.cpp` 一起编译也没用——它被编译了，但里面没有被用到的模板，依然什么都不生成（见本章第一幅图）。

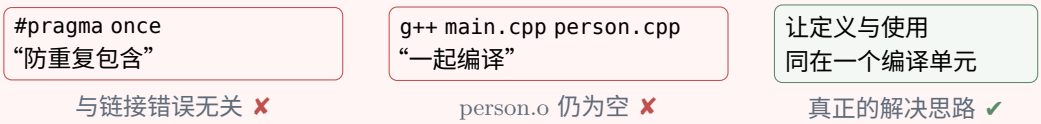


图 59　两种无效的“解决办法”与真正的思路

生活中的它 家具厂：图纸、订单与总装

生活场景：A 分厂有柜子图纸，但没接到订单，什么也没生产；B 分厂接到“做一个 120 cm 柜子”的订单，但手里只有目录、没有图纸，只能写一张“请从别处调货”的单子；总装车间拿着调货单去两个厂的仓库找，找不到柜子。

小判断：三种解决方法分别相当于什么？——方法 1、2：把图纸复印一份给 B 厂（B 厂自己做）；方法 3：提前通知 A 厂“先生产 120 cm 款”（只限这一款）。**边界**：工厂可以临时补救；链接器不会生成任何代码，所以必须在编译阶段就安排好。

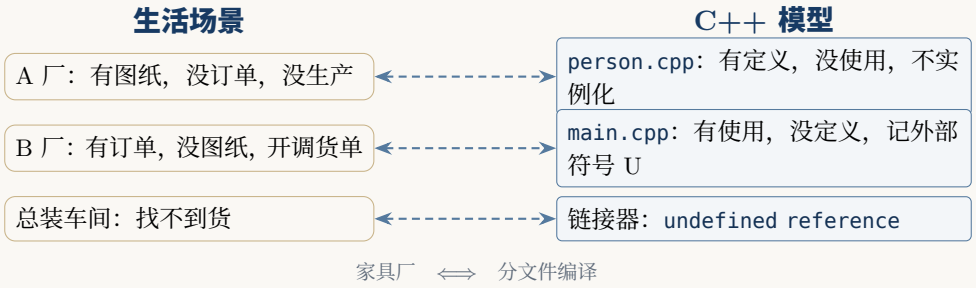


图 60　家具厂与分文件编译的对应

对照：普通函数的分文件↔模板的分文件（方法互补型，跨专题）

一句话定位：你在“封装”专题里学到的“声明放 `.h`、定义放 `.cpp`”是普通函数与普通类的标准做法；对模板，这条经验要整个翻过来。



图 61　对照图：普通函数的分文件经验能否搬到模板上

迁移判断：几乎全部“不成立”，这正是这个专题最容易出错的原因。第 3 行“加条件”就是原图纠错 5 的内容：模板实例化出的函数在多个编译单元中重复出现是被允许的（弱符号，链接器合并），只有混入普通函数时才会重复定义。**同一个小算例**：把 `int add(int,int)` 与 `template<class T> T add(T,T)` 分别按“`.h` + `.cpp`”拆开并在 `main` 中调用 `add(1,2)`——前者链接成功，后者 `undefined reference to` `` `int add<int>(int, int)' ``。

教学补全 历史回声：`export` 关键字 【拓展联想】

第 2 章提到，C++98 曾用 `export template` 允许模板像普通函数一样分开编译，但它要求编译器在链接阶段“回头”实例化，实现极其复杂，几乎无人实现，C++11 将其删除。所以今天的标准做法仍是方法 2。（C++20 的“模块”提供了新的组织方式，属于后续内容。）

学完这里，你应该能够回答

(1) 用一句话说出链接错误的成因。
(2) 三种方法各靠什么生效?
(3) 为什么 `#pragma once` 不能解决?

5.3　走向数据结构：第一个通用容器 MyStack<T>

把本专题的知识合在一起，就能写出数据结构课里的第一个“元素类型待定”的容器——栈。下面的代码放在一个 `mystack.hpp` 中（方法 2），用类外实现（K8），并且只在用到时实例化（K9）。【教学补全】

```
template <class T>
class MyStack {
public:
    MyStack(int cap) : m_Cap(cap), m_Size(0) { m_Data = new T[cap]; }
    ~MyStack() { delete[] m_Data; }
    bool push(const T &x);              // 入栈，满了返回 false
    bool pop();                         // 出栈，空了返回 false
    T top() const { return m_Data[m_Size - 1]; }
    bool empty() const { return m_Size == 0; }
```

```
private:
    T  *m_Data;   int m_Cap;   int m_Size;
};
template <class T>
bool MyStack<T>::push(const T &x) {
    if (m_Size == m_Cap) return false;   // 栈满
    m_Data[m_Size++] = x;                // 先放到 m_Size 位置，再把 m_Size 加 1
    return true;
}
template <class T>
bool MyStack<T>::pop() {
    if (m_Size == 0) return false;       // 栈空
    --m_Size;                            // 只需把“顶”往下移
    return true;
}
```

逐段解释：第 1–12 行是类模板，元素类型 `T` 待定，用动态数组 `m_Data` 存元素，`m_Size` 既是元素个数，也是下一个空位的下标；第 13–18 行 `push` 先判满，再存入；第 19–24 行 `pop` 先判空，再把栈顶下标减 1。

运行（已实测）：`MyStack<int> s(3);` 依次 `push` 10、20、30 后再 `push(40)` 返回 `0`（满），`top()` 为 `30`；`pop()` 后 `top()` 为 `20`。同一份代码 `MyStack<std::string> t(2);` 压入 `"A"`、`"B"` 后 `top()` 为 `"B"`。

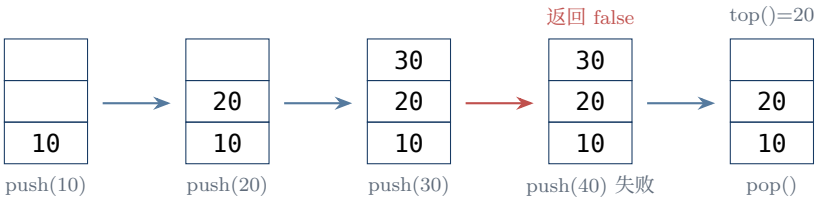


图 62　MyStack<int>（容量 3）的状态变化：底部为下标 0

注：为突出模板，本例省略了复制构造与赋值运算（不要复制 `MyStack` 对象）；`top()` 也未检查栈空。这些都会在数据结构课中补上。

> **知识联系 后续课程**
>
> 数据结构课中的顺序表 `SeqList<T>`、链表 `LinkList<T>`、队列、二叉树 `BinTree<T>`，写法都与 `MyStack<T>` 相同；STL 中的 `std::stack<T>` 就是一个更完善的版本。到那时你会发现：本专题的四个源文件，正好是写任何一个通用容器的四个步骤。

本章知识地图

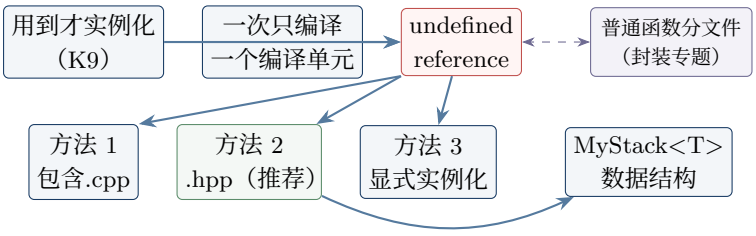


图 63　第 5 章知识地图

到这里，我们已经解决了什么？

从“为什么要写两遍 Swap”出发，我们走完了函数模板、类模板、类外实现、实例化时机、分文件编写，最后写出了一个通用的栈。接下来用 38 道题检查掌握情况：先做 25 道基础题，再做 10 道提升题，最后 3 道跨学科题；答案和解析都集中在后面，请先独立完成。

第 6 章　正式练习（38 题）

使用说明与知识点—题目对应表

练习分三层：基础 25 题、提升 10 题、跨学科 3 题，共 38 题。题目中的图只呈现题目已给出的信息；答案速查见第 7 章，详细解析（含解题图）见第 8 章。凡涉及代码运行结果的题，答案均已在 g++ 13 上实测。

核心知识点	基础题	提升题	跨学科	涉及的对照
K1 为什么需要模板	1, 11, 21	29		重载 / 宏
K2 函数模板语法	2, 3, 16, 21			函数族 $f_a(x)$
K3 实例化	4, 5, 11, 18	29, 32	37	宏、数域
K4 推导与显式指定	6, 7, 17, 22	26, 29, 31	38	待定系数法
K5 类型的隐含要求	12	26, 27, 31, 34	36, 37, 38	有序域 / 复数
K6 类模板	8, 13	32, 34		分布族
K7 对象作函数参数	14, 23	32		定义域
K8 类外实现	9, 19	30, 33, 34		普通类类外实现
K9 用到才实例化	20, 25	27, 35		Python 运行期报错
K10 分文件编写	10, 15, 24	28, 30, 35		普通函数分文件

以下各题中，凡未特别说明，均假定已 `#include <iostream>`、`<string>` 并 `using namespace std;`，编译标准为 C++14 或 C++17（第 8 题特别注明）。

6.1　A. 基础巩固训练（25 题）

基础 单项选择题（第 1–10 题）

1. 函数模板最主要的作用是（　）。

A. 提高程序的运行速度

B. 对“逻辑相同、只有类型不同”的代码只写一次，提高复用性

C. 让函数在程序运行时改变参数类型

D. 减少可执行文件中生成的函数个数

2. 下列函数模板声明中，正确的是（　）。

A. `template <T>`　　B. `template <typename T>`

C. `template (typename T)`　　D. `typename <template T>`

3. 在模板参数列表中，关键字 `typename` 与 `class` 的关系是（　）。

A. 完全等价，可以互换

B. `class` 只能用于类类型，不能填 `int`

C. `typename` 只能用于内置类型

D. 函数模板只能用 `typename`，类模板只能用 `class`

4. 下图是一个 C++ 程序从源代码到运行的四个阶段。函数模板的实例化发生在图中哪个位置？（　）

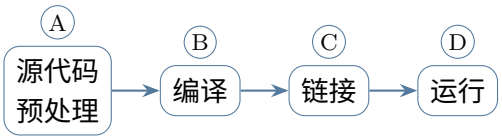


图 64　第 4 题题图：程序的四个阶段

A. A（预处理阶段）　　B. B（编译阶段）

C. C（链接阶段）　　D. D（运行阶段，第一次调用时）

5. 已知 `template<class T> void mySwap(T &a, T &b);`，`main` 中依次调用（变量类型标在下图中）：

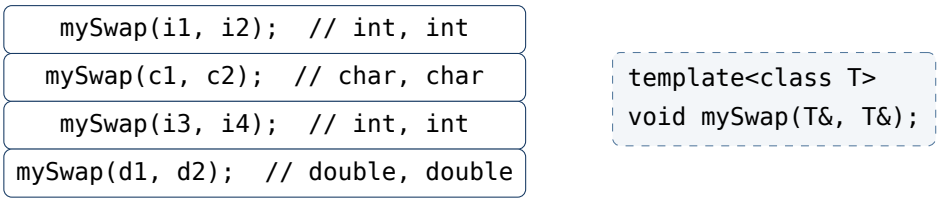


图 65　第 5 题题图：四次调用

编译器共生成 `mySwap` 的实例个数为（　）。

A. 1　　B. 2

C. 3　　D. 4

6. 已知 `template<class T> void mySwap(T &a, T &b);`，执行 `int a = 1; double c = 2.0; mySwap(a, c);`，结果是（　）。

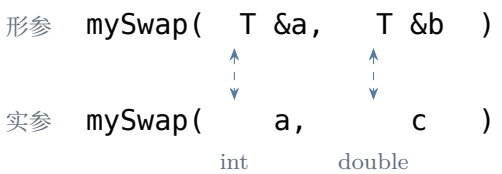


图 66　第 6 题题图：形参与实参的对齐

A. 编译通过，`T` 推导为 `int`

B. 编译通过，`T` 推导为 `double`

C. 编译错误：`T` 的推导结果冲突

D. 能编译，但运行时出错

7. 已知 `template<class T> void func() { cout << "func"; }`，下列调用正确的是（　）。



圆圈处：形参列表为空

图 67　第 7 题题图：`func` 的形参列表

A. `func();`　　B. `func<int>();`

C. `func(int);`　　D. `template func();`

8. 已有类模板 `template<class T1, class T2> class Person { public: Person(T1 n, T2 a); };`。在 **C++14** 标准下，能正确创建对象的语句是（　）。

A. `Person p("Tom", 20);`

B. `Person<string, int> p("Tom", 20);`

C. `Person<> p("Tom", 20);`

D. `Person(string, int) p("Tom", 20);`

9. 类模板 `Person<T1,T2>` 的成员函数 `void ShowPerson();` 在类内只有声明。下列类外实现正确的是（　）。

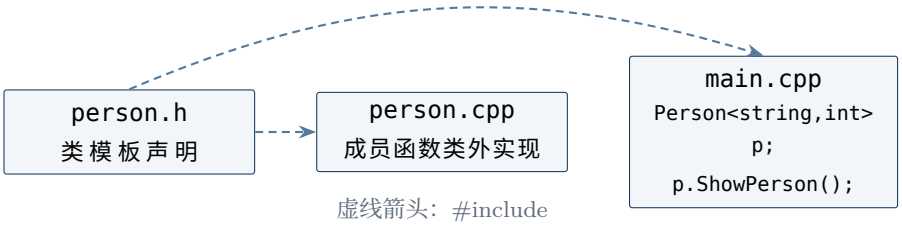
A. `void Person::ShowPerson() { }`

B. `template<class T1, class T2> void Person::ShowPerson() { }`

C. `void Person<T1, T2>::ShowPerson() { }`

D. `template<class T1, class T2> void Person<T1, T2>::ShowPerson() { }`

10. 把类模板按普通类的习惯拆成三个文件（见下图），用 `g++ main.cpp person.cpp` 生成程序，会出现（　）。



虚线箭头：#include

图 68　第 10 题题图：三文件结构

A. 预处理错误：找不到 `person.h`

B. 编译错误：`ShowPerson` 未声明

C. 链接错误：`undefined reference`

D. 程序运行时崩溃

基础 多项选择题（第 11–15 题）

11. 关于下图中的函数模板与调用，下列说法正确的有（　）。

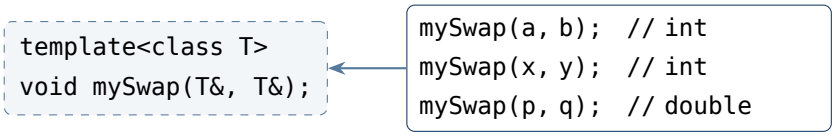


图 69　第 11 题题图

A. 模板本身不产生机器码

B. `mySwap(a,b)` 与 `mySwap(x,y)` 调用的是同一个实例

C. 一个模板可以有多个类型参数

D. 使用模板后，可执行文件中只有一个 `mySwap` 函数

12. 已知 `template<class T> T myMax(T a, T b) { return a < b ? b : a; }`。用下列类型实例化 `myMax`，能够成功编译的有（　）。

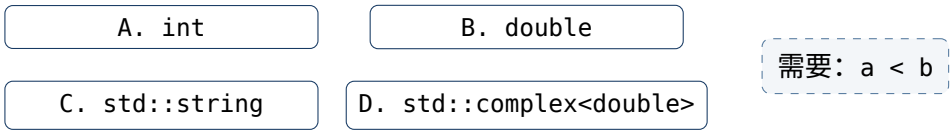


图 70　第 12 题题图：四种候选类型

A. `int`　　B. `double`

C. `std::string`　　D. `std::complex<double>`

13. 关于类模板 `template<class T1, class T2> class Person`，下列说法正确的有（　）。

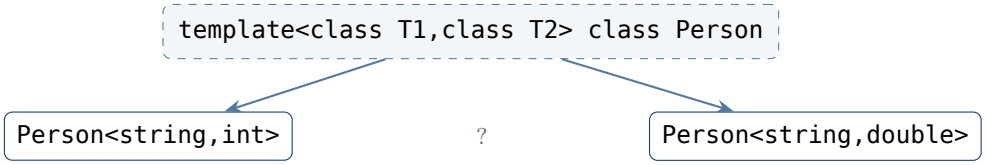


图 71　第 13 题题图

A. `Person` 单独出现时不是一个类型

B. `Person<string,int>` 与 `Person<string,double>` 是两种不同的类型

C. 类模板的类型参数可以有默认值，如 `class T2 = int`

D. `Person<string,int>` 对象与 `Person<string,double>` 对象可以直接用 `=` 互相赋值

14. 已有 `Person<string, double> q("Lee", 20.5);`。下列函数中，能接收 `q` 作为实参的有（　）。

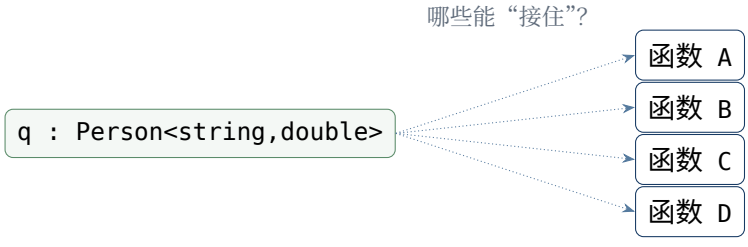


图 72　第 14 题题图

A. `void show(Person<string,int> &p);`

B. `template<class T1,class T2> void show(Person<T1,T2> &p);`

C. `template<class T> void show(T &p);`（函数体只访问 `p.m_Name`）

D. `void show(Person &p);`

15. 类模板按 `.h` + `.cpp` 分开写会出现链接错误。下列做法中，能够解决该问题的有（　）。

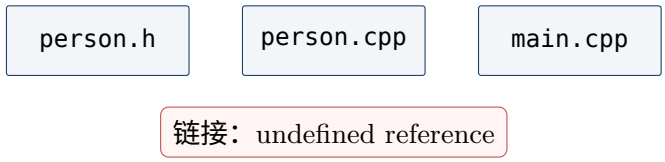


图 73　第 15 题题图

A. 把声明和实现合并为 `person.hpp`，`main.cpp` 包含它

B. `main.cpp` 中 `#include "person.cpp"`，只编译 `main.cpp`

C. 在 `person.cpp` 末尾写 `template class Person<string,int>;`

D. 在 `person.h` 开头加 `#pragma once`

基础 填空题（第 16–20 题）

16. 声明函数模板时使用的关键字是_____；尖括号中声明类型参数时，可以使用关键字_____ 或_____。

17. 已知 `template<class T> T myAdd(T a, T b) { return a + b; }`，则表达式 `myAdd<double>(1, 2.5)` 的值为_____。

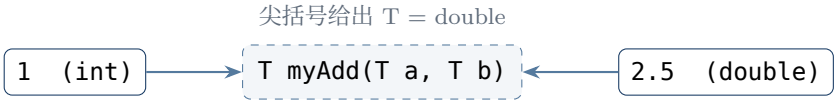


图 74　第 17 题题图

18. 执行 `int a = 3, b = 8; mySwap(a, b);`（`mySwap` 为 3.3 节的交换模板）后，`a` =_____，`b` =_____；被调用的实例记作_____。



图 75　第 18 题题图：调用前的状态

19. 把类模板 `Person<T1,T2>` 的构造函数写到类外。补全下面两行：

```
_____

_____::Person(T1 name, T2 age) { this->m_Name = name;
this->m_Age = age; }
```

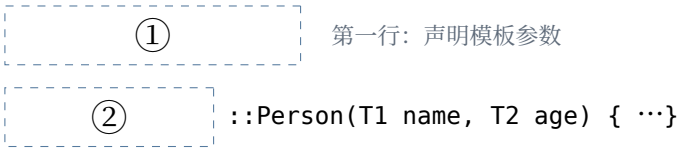


图 76　第 19 题题图：两个空分别所在的位置

20. 已知类模板 `Box<T>` 有成员函数 `show()`（输出 `v`）与 `half()`（返回 `v / 2`）。程序中只写了 `Box<string> b("hi"); b.show();`。则 `Box<string>::half()` _____（填“被”或“未被”）实例化，该程序_____（填“能”或“不能”）通过编译。

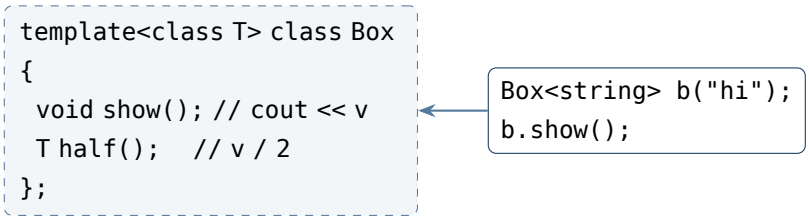


图 77　第 20 题题图

基础 解答题（第 21–25 题）

21. 下面是 函数模板.cpp 中被注释掉的 `SwapInt`。(1) 把它改写成函数模板 `MySwap`；(2) 指出改写时修改了哪几处，并说明为什么可以这样改。

```
1 void SwapInt(int &a, int &b) { int temp = a; a = b; b = temp; }
```

22. 已知 `template<typename T1, typename T2> void printPair(T1 x, T2 y) { cout << "(" << x << ", " << y << ")" << endl; }`。分别写出 (1) `printPair('A', 3.5);` (2) `printPair<int, int>('A', 3.5);` 中 `T1`、`T2` 的类型和输出结果（`'A'` 的编码为 65）。

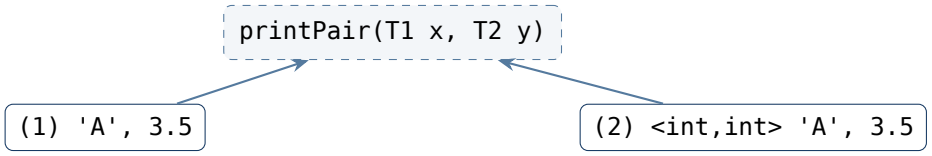


图 78　第 22 题题图：两种调用

23. 写一个函数模板 `Show2`，使它能打印**任意** `Person<T1,T2>` 对象的姓名与年龄；并写出调用 `Show2(r)`（`r` 为 `Person<int, double>`）时推导出的 `T1`、`T2`。

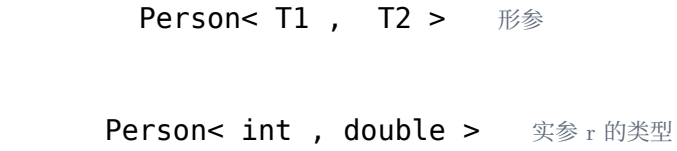


图 79　第 23 题题图

24. 某同学按下图把类模板拆成三个文件，`main.cpp` 中使用了 `Person<string,int>` 的构造函数与 `ShowPerson`。他用 `nm -C` 查看两个目标文件，得到图中信息。请根据这些信息说明：(1) 错误发生在哪个阶段、报什么错；(2) 为什么 `person.o` 中什么都没有；(3) 为什么 `main.o` 中是 `U`。

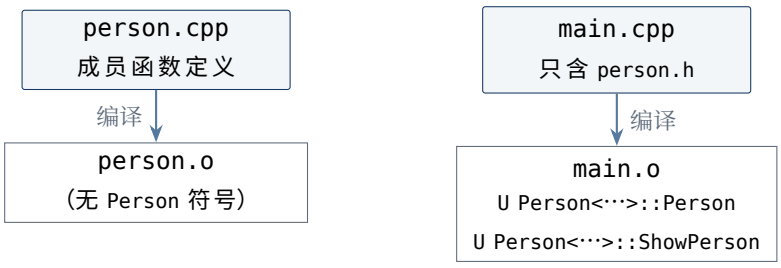


图 80　第 24 题题图：两个目标文件的符号表

25. 已知

```
template <class T> struct Box {
    T v;
    Box(T x) : v(x) {}
    void show() { cout << v << endl; }
    T half()    { return v / 2; }
};
int main() { Box<int> b(7); cout << b.half() << endl; Box<string> s("ab"); s.show
    (); }
```

(1) 程序能否编译？说明 `Box<int>` 与 `Box<string>` 各实例化了哪些成员函数；(2) 写出输出。

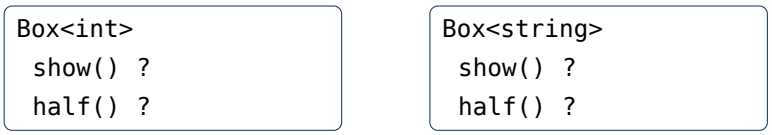


图 81　第 25 题题图：待判断的成员函数

6.2　B. 中等偏下提升训练（10 题）

提升　单项选择题（第 26–28 题）

26. 已知 `template<class T> T myMax(T a, T b) { return a < b ? b : a; }`。下列调用中，输出结果为 `7` 的是（　）。

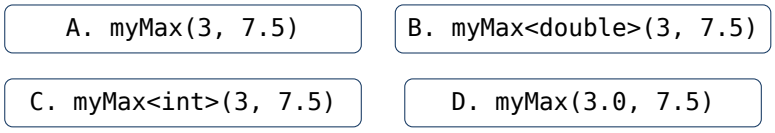


图 82　第 26 题题图：四个调用

A. `myMax(3, 7.5)`　　B. `myMax<double>(3, 7.5)`

C. `myMax<int>(3, 7.5)`　　D. `myMax(3.0, 7.5)`

27. 已知（`z1`、`z2` 为 `std::complex<double>`，标准库没有为复数定义 `<`）：

```
template <class T> class Pair {
public:
    Pair(T a, T b) : a_(a), b_(b) {}
    T bigger() { return a_ < b_ ? b_ : a_; }
    T a_, b_;
};
int main() { Pair<std::complex<double>> p(z1, z2); cout << p.a_; }
```

下列判断正确的是（　）。

A. 编译失败，因为 `complex` 没有 `<`，`Pair<complex<double>>` 无法生成

B. 能编译，因为 `bigger()` 没有被使用，不会被实例化

C. 能编译，因为标准库为 `complex` 定义了 `<`

D. 能编译，但链接时找不到 `bigger()`

28. `person.h` 只有类模板声明；`person.cpp` 有类外实现，并在末尾写了 `template class Person<string, int>;`；`main.cpp` 中依次使用了 `Person<string,int>` 与 `Person<string,double>` 的构造函数和 `ShowPerson`。执行 `g++ main.cpp person.cpp`，结果是（　）。

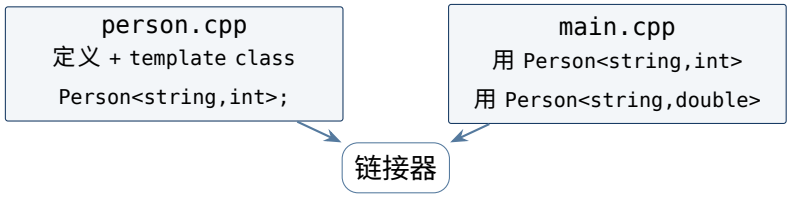


图 83　第 28 题题图

A. 链接成功，程序正常运行

B. 链接失败，且只缺 `Person<string,double>` 的成员函数

C. 编译失败，`Person<string,double>` 未声明

D. 链接失败，两种 `Person` 的成员函数都缺

提升 多项选择题（第 29–30 题）

29. 已知 `template<class T> void mySwap(T &a, T &b);`，`main` 中：

```
int a = 1, b = 2;  double x = 1.5, y = 2.5;
mySwap(a, b);              // ①
mySwap<double>(x, y);      // ②
mySwap(x, y);              // ③
```

下列说法正确的有（　）。

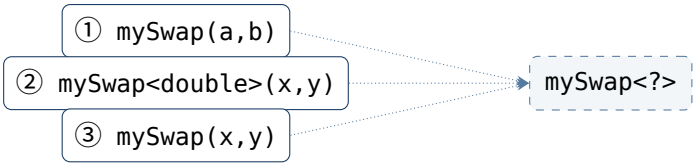


图 84　第 29 题题图

A. 这段代码共生成 2 个 `mySwap` 实例

B. ②与③调用的是同一个函数

C. 若再加一行 `mySwap<int>(a, x);`，程序仍能编译

D. 三次调用中 `T` 都是在编译期确定的

30. 某同学把类模板 `Person` 写成 `person.hpp`（声明与类外实现都在其中），并在 `a.cpp`、`main.cpp` 两个文件中都包含它、都使用 `Person<string,int>`，然后 `g++ a.cpp main.cpp`。下列说法正确的有（　）。

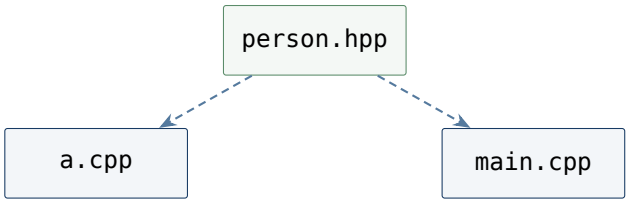


图 85　第 30 题题图

A. `person.hpp` 中每个类外实现的成员函数前都必须写 `template<class T1,class T2>`

B. `.hpp` 后缀是编译器强制要求的，改成 `.h` 就会出错

C. 两个编译单元都会实例化 `Person<string,int>` 的成员函数，但链接时不会报重复定义

D. 既然用了 `.hpp`，类外实现可以简写成 `void Person::ShowPerson()`

提升 填空题（第 31–33 题）

31. 已知 `template<class T> T avg(T a, T b) { return (a + b) / 2; }`，则 `avg(3, 4)` 的值为_____，`avg(3.0, 4.0)` 的值为_____，`avg<double>(3, 4)` 的值为_____。

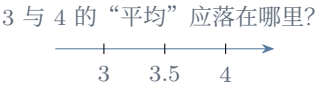


图 86　第 31 题题图

32. 已知 `template<class T1, class T2> void Show2(Person<T1,T2> &p);`，程序中有 `Person<int,double> q(7, 1.5); Person<int,int> r(1, 2);`，并依次调用 `Show2(q); Show2(q); Show2(r);`。则调用 `Show2(q)` 时 `T1 =`_____，`T2 =`_____；整个程序共生成 `Show2` 的实例_____ 个。

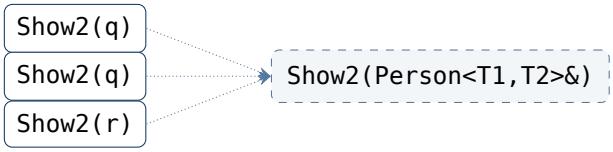


图 87　第 32 题题图

33. 对 5.3 节的 `MyStack<int> s(2);` 依次执行 `s.push(5); s.push(6); s.push(7); s.pop(); s.push(8);`。则 `s.push(7)` 的返回值为_____（填 `true`/`false`），最终 `s.top()` 为_____，最终 `m_Size` 为_____。



初始：空，容量 2

push(5) → push(6) → push(7) → pop() → push(8)

图 88　第 33 题题图：初始状态与操作序列

提升 综合大题（第 34–35 题）

34. 已知类模板（成员函数 `sum` 只在类内声明）：

```
template <class T>
class Pair {
public:
    Pair(T a, T b) : a_(a), b_(b) {}
    T sum();
    T a_, b_;
};
```

(1) 写出 `sum()`（返回 `a_ + b_`）的类外实现；

(2) 写出 `Pair<int> p(3, 4); p.sum()` 与 `Pair<string> s("ab", "cd"); s.sum()` 的结果；

(3) `Pair<double> d(1, 2.5);` 能否编译？若能，`d.sum()` 为多少？

(4) 若再给 `Pair` 增加成员 `T bigger() { return a_ < b_ ? b_ : a_; }`，`Pair<std::complex<double>>` 的对象还能否创建并调用 `sum()`？说明理由。

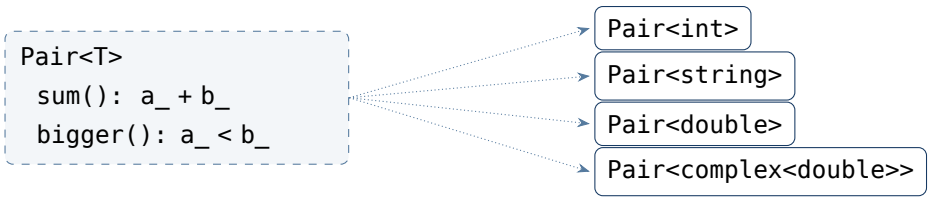


图 89　第 34 题题图：一个类模板，四种实参

35. 某工程有三个文件：`person.h`（类模板 `Person<T1,T2>` 声明）、`person.cpp`（构造函数与 `ShowPerson` 的类外实现）、`main.cpp`（`#include "person.h"`）。

(1) 若 `main` 中只有 `Person<string,int> p("Tom", 20);`（没有调用 `ShowPerson`），`g++ main.cpp person.cpp` 会报几条 `undefined reference`？分别缺什么？

(2) 用“显式实例化”修复 (1)，写出需要添加的语句及位置；

(3) 修复后，若 `main` 又使用了 `Person<string,double>`，会发生什么？

(4) 改用“合并为 `.hpp`”的方法，写出 `person.hpp` 的结构（可只写骨架），并说明它为什么对任意类型都有效。

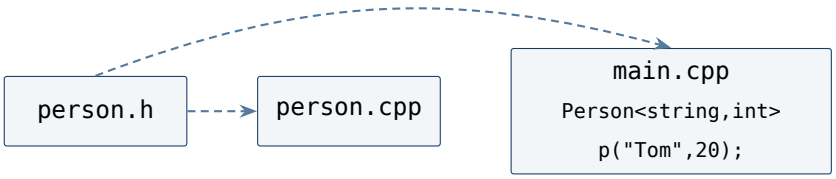


图 90　第 35 题题图

6.3　C. 跨学科迁移训练（3 题）

36. 跨学科 **（高等代数 / 复变函数 × 模板，单选）**背景：复数集 $\mathbb{C}$ 上不能定义与加法、乘法相容的大小顺序，所以 C++ 标准库没有为 `std::complex<double>` 定义 `<`；但 `std::abs(z)` 返回复数 z 的模 $|z|$（一个 `double`）。设 `z1` $= 1+2i$，`z2` $= 3$，`myMax` 同第 12 题。下列语句中**能够通过编译**的是（　）。

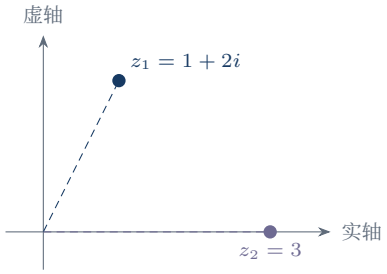


图 91　第 36 题题图：两个复数在复平面上的位置

A. `myMax(z1, z2);`

B. `myMax(std::abs(z1), std::abs(z2));`

C. `myMax<std::complex<double>>(z1, z2);`

D. `myMax(z1, 3.0);`

37. 跨学科 **（自动控制 / 数字信号处理 × 模板，多选）**背景：一阶数字低通滤波器每一步按 $y_k = a\,y_{k-1} + (1-a)\,x_k$ 更新输出（$0 < a < 1$，a 越大越“平滑”）。用模板实现：

```
template <class T> T lowpass(T yprev, T x, T a) { return a * yprev + (1 - a) * x;
    }
```

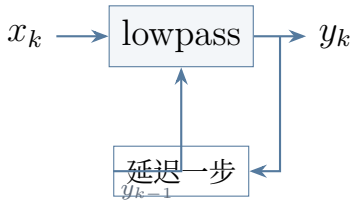


图 92　第 37 题题图：一阶低通滤波器的结构

下列说法正确的有（　）。

A. `lowpass<double>(0, 10, 0.8)` 的结果约为 `2`

B. `lowpass<int>(0, 10, 0.8)` 中 `a` 被截断为 `0`，结果为 `10`，滤波作用完全消失

C. `lowpass<float>` 与 `lowpass<double>` 是两个不同的函数实例

D. 因为 `T` 是模板参数，程序可以在运行时根据传感器精度把 `T` 从 `float` 切换成 `double`

38. 跨学科 **（大学物理 × 模板，填空）**背景：质点动能 $E_k = \frac{1}{2}mv^2$。用模板实现 `template <class T> T kinetic(T m, T v) { return m * v * v / 2; }`。则 `kinetic(3, 3)` 的值为_____，`kinetic(3.0, 3.0)` 的值为_____；若质量 $m = 3\,\mathrm{kg}$、速度 $v = 3\,\mathrm{m/s}$，动能的正确值为_____J。

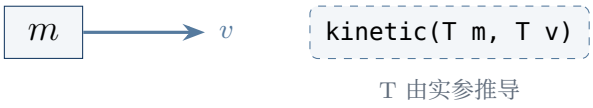


图 93　第 38 题题图

第 7 章　答案速查

本章只给结果，过程见第 8 章。

A. 基础巩固（25 题）

单选　1. B　2. B　3. A　4. B　5. C　6. C　7. B　8. B　9. D　10. C

多选　11. ABC　12. ABC　13. ABC　14. BC　15. ABC

填空

16. `template`；`typename`；`class`（后两空顺序可交换）　17. `3.5`　18. `8`；`3`；`mySwap<int>`

19. `template <class T1, class T2>`（`class` 写成 `typename` 亦可）；`Person<T1, T2>`　20. 未被；能

解答

21. `template <typename T> void MySwap(T &a, T &b) { T temp = a; a = b; b = temp; }`；改 3 处类型 + 函数名。

22. (1) `T1=char, T2=double`，输出 `(A, 3.5)`；(2) `T1=int, T2=int`，输出 `(65, 3)`。

23. `template<class T1,class T2> void Show2(Person<T1,T2> &p){…};` `T1=int, T2=double`。

24. 链接阶段，`undefined reference`；`person.cpp` 中无人使用不实例化；`main.cpp` 看不到定义只能留外部符号。

25. 能编译；`Box<int>`：构造函数、`half`；`Box<string>`：构造函数、`show`；输出 `3` 换行 `ab`。

B. 中等偏下提升（10 题）

单选　26. C　27. B　28. B　　**多选**　29. ABD　30. AC

填空　31. `3`；`3.5`；`3.5`　32. `int`；`double`；`2`　33. `false`；`8`；`2`

综合

34. (1) `template <class T> T Pair<T>::sum() { return a_ + b_; }`；(2) `7`；`"abcd"`；(3) 能，`3.5`；(4) 能，`bigger()` 未被使用、不实例化。

35. (1) 1 条，只缺 `Person<string,int>::Person(string,int)`；(2) 在 `person.cpp` 末尾加 `template class Person<std::string, int>;`；(3) 链接时缺 `Person<string,double>` 的成员函数；(4) 声明与类外实现合写进 `person.hpp`，使用者能看到定义，可就地实例化任意类型。

C. 跨学科迁移（3 题）

36. B（结果为 3）　　37. ABC　　38. `13`；`13.5`；`13.5`

第 8 章　详细解析

每题依次给出：答案与难度、考查知识点、解析图（适合画图的题）、思路与过程、选项逐一判断（选择题）、常见错误与一句话总结。选择题中凡选项本身能画出来的，都配了逐选项小图：绿色为正确，红色为错误。

8.1　基础单选题解析（第 1–10 题）

第 1 题　答案：B　难度：基础

考查：K1 为什么需要模板。**依据**：3.1 节定义——泛型编程把与类型无关的逻辑写一次。

逐项：A 错，模板生成的代码和手写的一样快，但不会“更快”，它不是为提速而设计的。B 对。C 错，`T` 在编译期确定（3.3 节），运行时不能改变。D 错，用到几种类型就生成几个实例，生成的函数个数并没有减少（3.1 节例题 1 右图）。

常见错误：把“少写代码”误解为“少生成代码”而选 D。　**一句话**：模板省的是人写的代码。

第 2 题　答案：B　难度：基础

考查：K2 语法。正确格式是 `template <typename T>`：关键字 `template` + 尖括号 + “`typename`/`class` + 名字”。

逐项：A 漏了 `typename`，编译器不知道 `T` 是类型名；C 用了圆括号；D 关键字顺序颠倒。　**一句话**：先 `template`，再尖括号，括号里先说“是类型”再给名字。

第 3 题　答案：A　难度：基础

考查：K2 `typename` 与 `class`。在模板参数列表中二者完全等价（原图纠错 3）。

逐项：B 错，`class T` 也可以填 `int`；C 错；D 错，源文件 `函数模板.cpp` 中就同时用了 `typename T` 和 `class M`。　**一句话**：参数列表里写哪个都一样。

第 4 题　答案：B　难度：基础

考查：K3 实例化发生在编译期。



图 94　第 4 题解析图：四个阶段逐一判断

图上证据：生成机器码需要知道 `T` 的大小与运算方式（3.3 节论证），只有编译器能做；链接器“只找不造”，这一点在第 5 章导致了链接错误。　**常见错误**：选 D，把实例化和函数调用混为一谈。　**一句话**：实例化在编译期，调用在运行期。

第 5 题　答案：C　难度：基础

考查：K3 实例的个数 = 不同类型组合的个数。

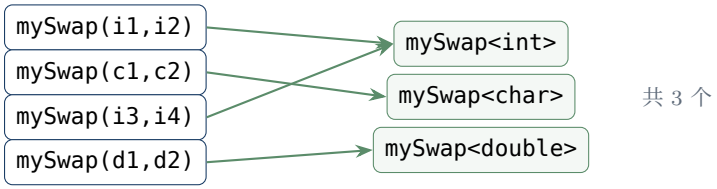


图 95　第 5 题解析图：4 次调用对应 3 个实例

逐项：A 把模板本身当成一个函数；B 漏数了 `char`；D 按调用次数计数。　**一句话**：数类型组合，不数调用次数。

第 6 题　答案：C　难度：基础

考查：K4 推导规则 2、3。

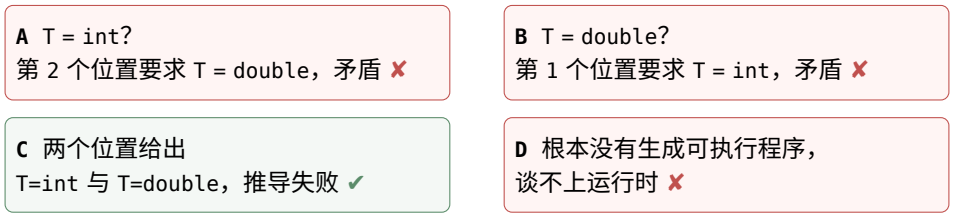


图 96　第 6 题解析图：逐选项判断

过程：对齐第 1 个形参得 `T=int`，对齐第 2 个得 `T=double`；推导不做隐式转换，冲突即失败。实测：`deduced conflicting types for parameter 'T' ('int' and 'double')`。　**常见错误**：以为编译器会“自动把 int 转成 double”而选 B。　**一句话**：推导要求精确一致。

第 7 题　答案：B　难度：基础

考查：K4 规则 4——`T` 不出现在形参中就无法推导。



图 97　第 7 题解析图

逐项：A 报 `couldn't deduce template parameter 'T'`；C 不是合法的调用语法（不能把类型当实参传）；D 不是调用语法。　**一句话**：推不出来就显式写。

第 8 题　答案：B　难度：基础

考查：K6 使用规则 1、4。C++14 中类模板必须写全模板实参。

逐项：A 在 C++14 报 `missing template arguments before 'p'`（C++17 可编译，但推出的是 `Person<const char*,int>`，见易错 7）；C 空尖括号只在所有参数都有默认值时才可用，本题没有默认值；D 不是合法语法。　**一句话**：初学阶段总是写全 `Person<string,int>`。

第 9 题　答案：D　难度：基础

考查：K8 类外实现的三个组成部分：① `template` 行；② `Person<T1,T2>::`；③函数名。

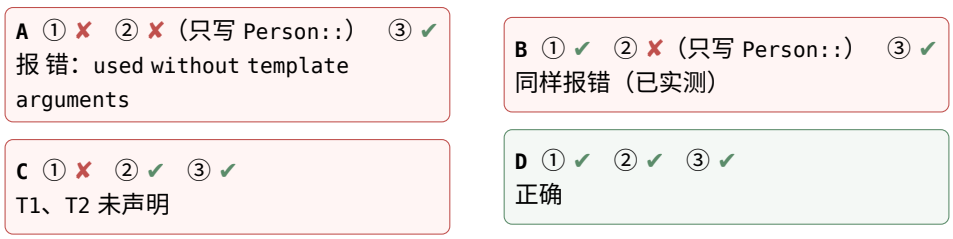


图 98　第 9 题解析图：每个选项缺了哪一部分

一句话：比普通类多两样东西，一个都不能少。

第 10 题　答案：C　难度：基础

考查：K10。



图 99　第 10 题解析图：错误发生在哪一站

过程：`person.cpp` 有定义但没人用，不实例化；`main.cpp` 有使用但看不到定义，留下外部符号；链接器两边都找不到（5.1 节论证）。**逐项**：A 文件存在；B `ShowPerson` 在 `person.h` 中已声明；D 链接失败就没有可执行文件。　**一句话**：模板分文件的错误是链接错误，不是编译错误。

8.2　基础多选题解析（第 11–15 题）

第 11 题　答案：ABC　难度：基础

考查：K1、K3。

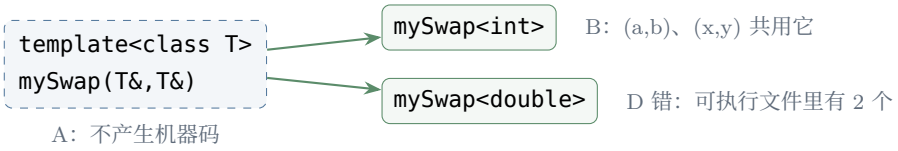


图 100　第 11 题解析图

逐项：A 对（3.3 节事实③）；B 对，二者都推出 `T=int`（事实②）；C 对，如源文件的 `template <typename T, class M>`；D 错，图中可见生成了 2 个实例。　**一句话**：一张图纸，按类型组合各出一份。

第 12 题　答案：ABC　难度：基础

考查：K5 类型要求——`myMax` 要求 `T` 支持 `<`。

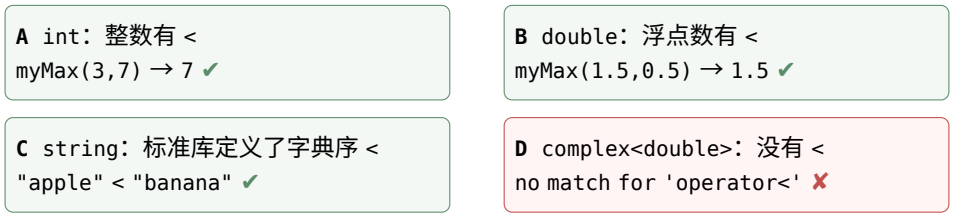


图 101　第 12 题解析图：四种类型逐一检查

图上证据：D 中复数在复平面上是“点”，无法排成一条有序的线（3.5 节对照）。**常见错误**：以为 `string` 不能比较而漏选 C。　**一句话**：函数体用到什么，类型就得有什么。

第 13 题　答案：ABC　难度：基础

考查：K6。

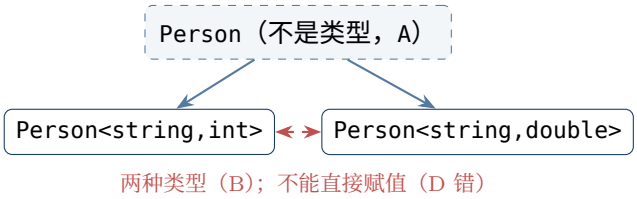


图 102　第 13 题解析图

逐项：A 对；B 对，内存布局都可能不同（4.1 节论证）；C 对，如 `template<class T1, class T2 = int>`；D 错，实测 `no match for 'operator='`。　**一句话**：同一张图纸，造出不同的类。

第 14 题　答案：BC　难度：基础

考查：K7 三种写法的接收范围。

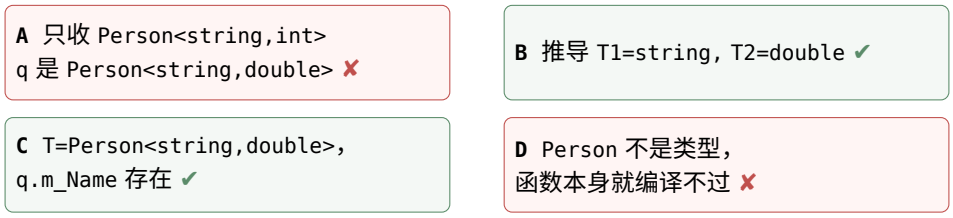


图 103　第 14 题解析图：每个函数能否“接住”q

一句话：形参写得越具体，接收范围越小。

第 15 题　答案：ABC　难度：基础

考查：K10 三种解决方法。

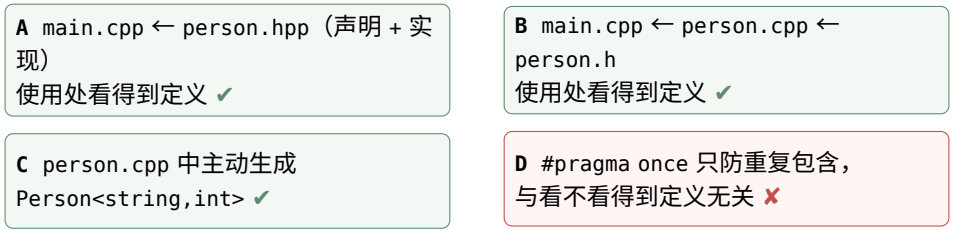


图 104　第 15 题解析图

注意：C 只对列出的 `Person<string,int>` 有效。　**一句话**：让定义与使用相遇，或在定义处主动生成。

8.3　基础填空题解析（第 16–20 题）

第 16 题　答案：template；typename；class　难度：基础

考查：K2。`template` 宣布“后面是模板”；尖括号里用 `typename` 或 `class` 说明紧跟的名字是类型。后两空顺序可交换。　**易错**：把 `typename` 写成 `typedef`。

第 17 题　答案：3.5　难度：基础

考查：K4 规则 5。

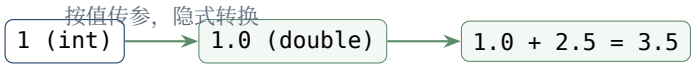


图 105　第 17 题解析图

过程：尖括号给出 `T=double`，不再推导；形参按值传递，`1` 隐式转换为 `1.0`；结果 `3.5`（已实测）。　**易错**：以为仍会“推导冲突”。若写成 `myAdd(1, 2.5)` 才会冲突。

第 18 题　答案：8；3；mySwap<int>　难度：基础

考查：K3 交换的状态追踪与实例名。

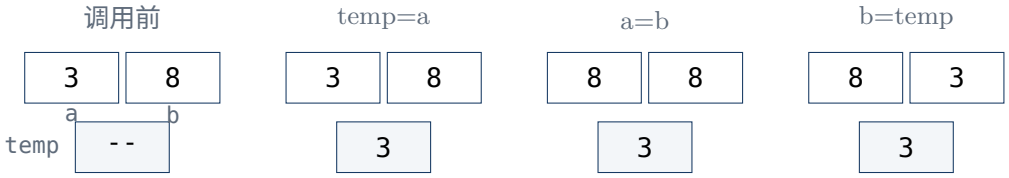


图 106　第 18 题解析图：三步交换

实例名：推导出 `T=int`，实例记作 `mySwap<int>`。

第 19 题　答案：template <class T1, class T2>；Person<T1, T2>　难度：基础

考查：K8。

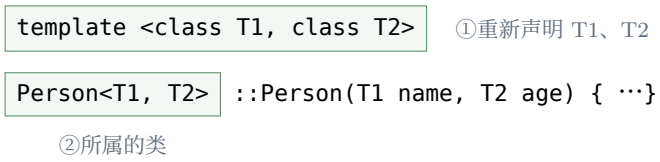


图 107　第 19 题解析图

构造函数没有返回类型，所以 `Person<T1,T2>::` 直接写在行首。　**易错**：第二空只写 `Person`。

第 20 题　答案：未被；能　难度：基础

考查：K9。

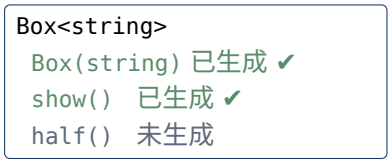


图 108　第 20 题解析图

`half()` 没有被调用，不实例化，因此虽然 `string / 2` 不存在，程序仍能编译，输出 `hi`（已实测）。

8.4　基础解答题解析（第 21–25 题）

第 21 题　答案：见下　难度：基础

题意：把只能交换 `int` 的函数改成对任意类型都可用的模板。**知识定位**：K1、K2。

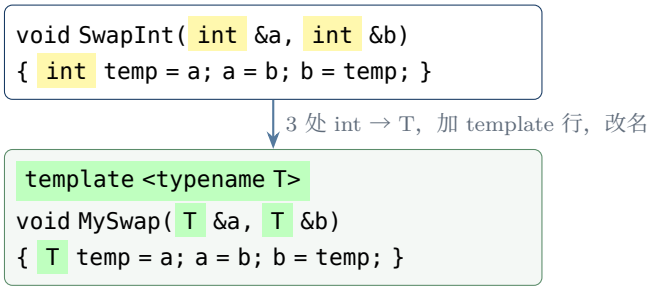


图 109　第 21 题解析图：改写前后对照

第一步：在函数前加 `template <typename T>`，声明类型参数。**第二步**：把两个形参和 `temp` 的类型 `int` 改为 `T`（共 3 处），函数名改为 `MySwap`。

为什么可以：交换算法只用到“复制”和“赋值”，不依赖“是整数”（3.1 节论证、3.3 节状态追踪），所以对任何可复制、可赋值的类型都成立。

检查：`MySwap<int>` 与原 `SwapInt` 完全相同。　**同类题**：先找出“随类型变化”的所有位置，再统一换成类型参数。

第 22 题　答案：(1) char, double, (A, 3.5)；(2) int, int, (65, 3)　难度：基础

知识定位：K4 推导与显式指定（规则 5）。

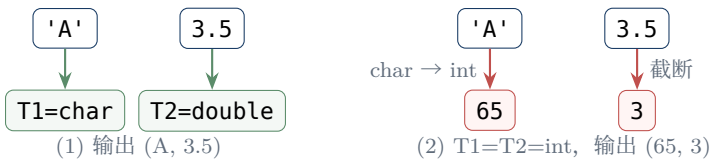


图 110　第 22 题解析图：推导与显式指定的不同结果

(1)：自动推导，`'A'` 是 `char`，`3.5` 是 `double`；输出 `(A, 3.5)`。(2)：显式指定两者都是 `int`，按值传参允许隐式转换：`'A'` 变为 `65`，`3.5` 截断为 `3`；输出 `(65, 3)`。两结果均已实测。

易错：以为显式指定后仍按 `char` 输出字母。　**一句话**：显式指定会“强行”把实参转成指定类型。

第 23 题　答案：见下　难度：基础

知识定位：K7 写法②。

```
template <class T1, class T2>
void Show2(Person<T1, T2> &p)
{
    cout << "姓名：" << p.m_Name << " 年龄：" << p.m_Age << endl;
}
```

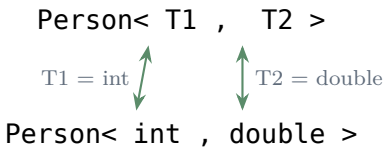


图 111　第 23 题解析图

推导：模板名 `Person` 对 `Person`，尖括号逐位对齐：`T1=int`，`T2=double`。　**易错**：写成 `void Show2(Person &p)`（易错 9）。

第 24 题　答案：链接阶段，undefined reference　难度：基础

知识定位：K9、K10。

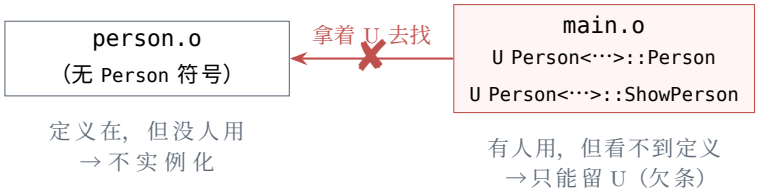


图 112　第 24 题解析图

(1)　编译两个文件都成功，**链接**时报 `undefined reference to Person<string,int>::Person(...)` 与 `...::ShowPerson()`。**(2)** 编译 `person.cpp` 时没有任何代码使用 `Person<string,int>`，按“用到才实例化”，什么都不生成。**(3)** 编译 `main.cpp` 时只看到声明，无法实例化，只能把它们当作外部符号（U）留给链接器。

同类题：遇到模板相关的 `undefined reference`，先问“定义和使用是否在同一个编译单元”。

第 25 题　答案：能编译；输出 3 和 ab　难度：基础

知识定位：K9。

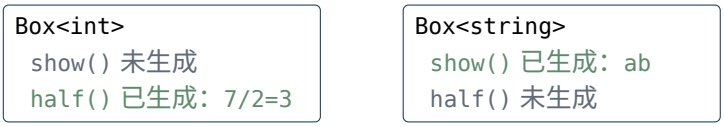


图 113　第 25 题解析图（两者的构造函数均已生成）

(1) 能编译。`Box<int>` 实例化了构造函数和 `half`；`Box<string>` 实例化了构造函数和 `show`。`Box<string>::half` 若生成会因 `string / 2` 出错，但它没被用到。**(2)** `7 / 2` 是整数除法，得 `3`；输出 `3` 换行 `ab`（已实测）。

易错：误以为 `7/2` 得 `3.5`——`T=int` 时是整数除法。

8.5　提升题解析（第 26–35 题）

第 26 题　答案：C　难度：中等偏下

考查：K4（推导、显式指定、隐式转换）+ K5。**知识链**：判断能否推导 → 确定 `T` → 计算转换后的实参 → 比较。

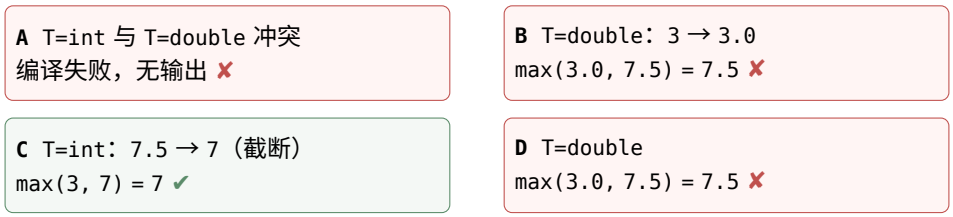


图 114　第 26 题解析图：四个调用分别发生了什么

图上证据：C 中红色的截断步骤是得到 `7` 的唯一原因（g++ 会给出 `-Wconversion` 警告）。　**最容易卡住**：以为 A 会自动转为 `double`。　**一句话**：显式指定 `int` 会悄悄截断小数。

第 27 题　答案：B　难度：中等偏下

考查：K5 + K9。

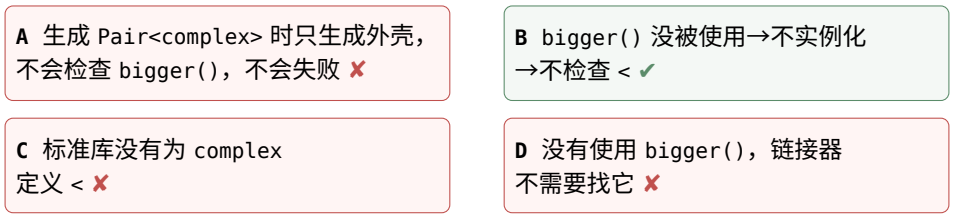


图 115　第 27 题解析图

一句话：不用的成员不生成，类型不必满足它的要求——这正是 STL 能容纳“只满足部分要求”的类型的原因。

第 28 题　答案：B　难度：中等偏下

考查：K10 方法 3 的适用范围。

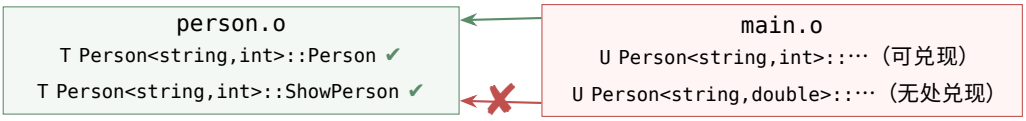


图 116　第 28 题解析图：两类欠条，一类能兑现

过程：显式实例化只生成了 `Person<string,int>`；`Person<string,double>` 无人生成。实测仅报两条：`undefined reference to Person<string, double>::Person(string, double)` 与 `...::ShowPerson()`。**逐项**：A 漏了 `double` 版；C 声明在 `person.h` 中，编译没问题；D `int` 版已被显式实例化。　**一句话**：显式实例化“点什么做什么”。

第 29 题　答案：ABD　难度：中等偏下

考查：K3 + K4（推导、显式、引用绑定）。

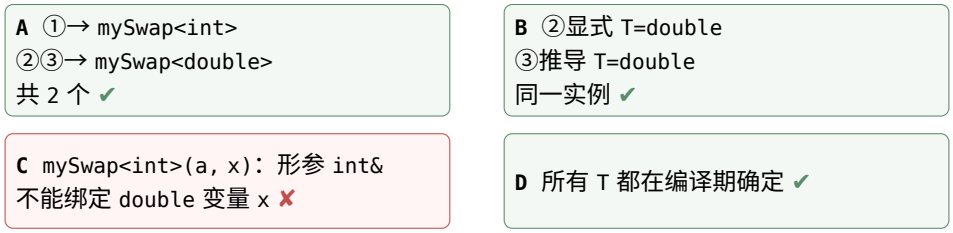


图 117　第 29 题解析图

C 的证据：与易错 5 相同，实测报 `cannot bind non-const lvalue reference`。**一句话**：显式指定能让按值参数转换，不能让引用参数转换。

第 30 题　答案：AC　难度：中等偏下

考查：K8 + K10。

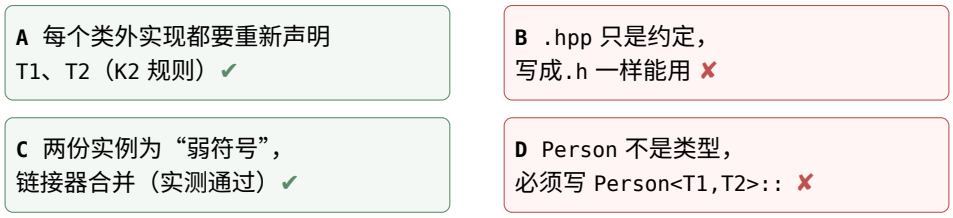


图 118　第 30 题解析图

C 的补充：这正是“原图纠错 5”的结论——模板实例化出的函数允许在多个编译单元中重复出现。实测 `g++ a.cpp main.cpp` 成功运行。　**一句话**：`.hpp` 改变的是“谁看得到定义”，不改变语法。

第 31 题　答案：3；3.5；3.5　难度：中等偏下

知识链：推导 `T` → 判断 `/` 是整数除法还是浮点除法 → 计算。



图 119　第 31 题解析图：整数除法把 3.5 截到 3

过程：`avg(3,4)` 推出 `T=int`，`(3+4)/2 = 7/2 = 3`；`avg(3.0,4.0)` 推出 `T=double`，得 `3.5`；`avg<double>(3,4)` 显式 `T=double`，`3`、`4` 转为 `3.0`、`4.0`，得 `3.5`（均已实测）。　**一句话**：同一个模板，结果取决于 `T`。

第 32 题　答案：int；double；2　难度：中等偏下

知识链：K7 推导 → K3 计数。

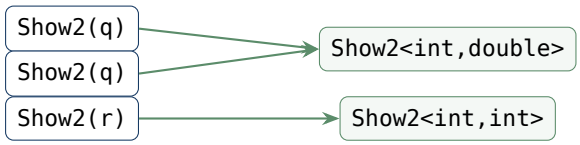


图 120　第 32 题解析图

`q` 为 `Person<int,double>`，逐位对齐得 `T1=int`，`T2=double`；`r` 得 `T1=T2=int`。不同组合 2 种，生成 2 个实例。

第 33 题　答案：false；8；2　难度：中等偏下

知识链：读懂 `push`/`pop` 的类外实现（K8）→ 逐步追踪状态。

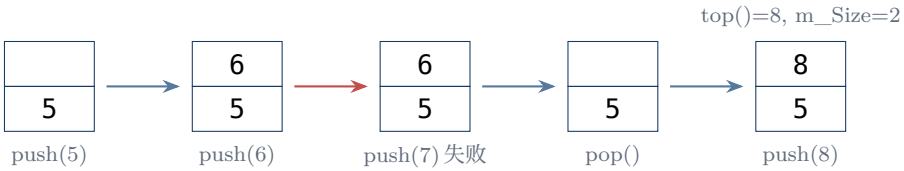


图 121　第 33 题解析图：容量为 2 的栈

过程：两次 `push` 后 `m_Size=2=m_Cap`，`push(7)` 判满返回 `false`，栈不变；`pop` 后 `m_Size=1`，栈顶为 5；`push(8)` 存到下标 1，`m_Size=2`，`top()=8`。　**易错**：以为 `pop` 弹出的是 7（7 从未入栈）。

第 34 题　答案：见下　难度：中等偏下 • 综合

1. 题目在问什么：一个类模板 `Pair<T>` 在不同 `T` 下的类外实现、计算结果与可用性。**2. 知识链**：K8 类外实现 → K6 实例化与构造 → K5 类型要求 → K9 用到才实例化。

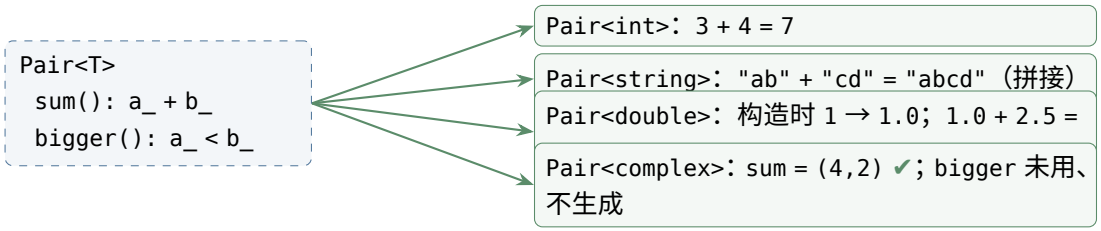


图 122　第 34 题解析图：一个类模板、四种实参的结果

(1)

```
template <class T>
T Pair<T>::sum() { return a_ + b_; }
```

为什么这样写：返回类型 `T` 写在 `template` 行之后；`Pair<T>::` 说明所属的类（K8 三个组成部分）。

(2) `Pair<int>`：`3 + 4 = 7`；`Pair<string>`：`string` 的 `+` 是拼接，得 `"abcd"`。

(3) 能。`T=double` 由尖括号给出，构造函数按值接收 `double`，`1` 隐式转换为 `1.0`；`sum()` 为 `3.5`。

(4) 能。`complex` 定义了 `+`，`sum()` 可实例化，$(1+2i)+(3+0i)=4+2i$，输出 `(4,2)`；`bigger()` 需要 `<`，`complex` 没有，但只要不调用就不实例化（K9）。

8. 结果检查：四个结果均已实测。**9. 最容易卡住**：(4) 以为“类型不支持 `<` 就整个类都不能用”。**10. 条件变化**：若在 (4) 中调用 `p.bigger()`，立刻编译失败（`no match for 'operator<'`）。

变式思考（不计题数）：若 `T=char`，`Pair<char> c('a','b'); c.sum()` 的结果还是“字母”吗？（提示：`char + char` 的结果类型是 `int`，再转回 `char`。）

第 35 题　答案：见下　难度：中等偏下 • 综合

1. 题目在问什么：分文件链接错误的“精确范围”及两种修复方法的差别。**2. 知识链**：K9 用到才实例化 → K10 成因 → 方法 3 → 方法 2。

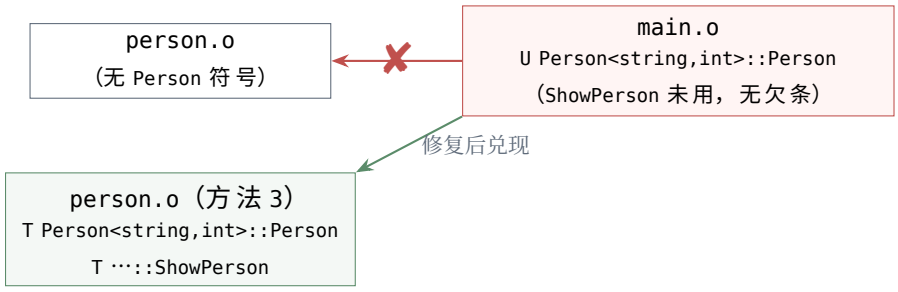


图 123　第 35 题解析图：只有被用到的成员才产生“欠条”

(1)　1　条。`main`　只用了构造函数，所以　`main.o`　只有一张欠条：`Person<string,int>::Person(string,int)`；`ShowPerson`　没被使用，不产生

欠条（K9，已实测只报 1 条）。

(2) 在 person.cpp 的成员函数定义之后（文件末尾）加：template class Person<std::string, int>;。它在看得到定义的地方强制生成全部成员。

(3) Person<string,double> 没有被显式实例化，链接时报它的 undefined reference（同第 28 题）。

(4)

```cpp
// person.hpp
#pragma once
#include <iostream>
#include <string>
template <class T1, class T2>
class Person {
public:
    Person(T1 name, T2 age);
    void ShowPerson();
    T1 m_Name;  T2 m_Age;
};
template <class T1, class T2>
Person<T1, T2>::Person(T1 name, T2 age) { m_Name = name; m_Age = age; }
template <class T1, class T2>
void Person<T1, T2>::ShowPerson() { std::cout << m_Name << " " << m_Age << std
    ::endl; }
```

为什么对任意类型有效：main.cpp 包含 person.hpp 后，同一个编译单元里同时有“定义”和“使用”，用到哪种类型就当场实例化哪种。

9. 最容易卡住：(1) 以为两个成员都会报错。**10. 条件变化**：若再调用 p.ShowPerson()，(1) 变为 2 条。

变式思考(不计题数):方法 3 中,如果想让 main.cpp 不再为 Person<string,int> 做任何实例化尝试，可以写什么？（提示：C++11 的 extern template class Person<std::string,int>;。）

8.6 跨学科迁移题解析（第 36–38 题）

第 36 题 答案：B 难度：跨学科・基础

当前专题知识：K5 类型要求（myMax 需要 <）。**借用的外部情境**：复数没有大小顺序、复数的模。**为什么不算超纲**：题干已给出“complex 没有 <”“abs 返回模长（double）”，不需要复变函数知识。**迁移点**：数学中“$\mathbb{C}$ 不能比大小，但模长可以比”搬到“complex 不能实例化 myMax，但 double 可以”。

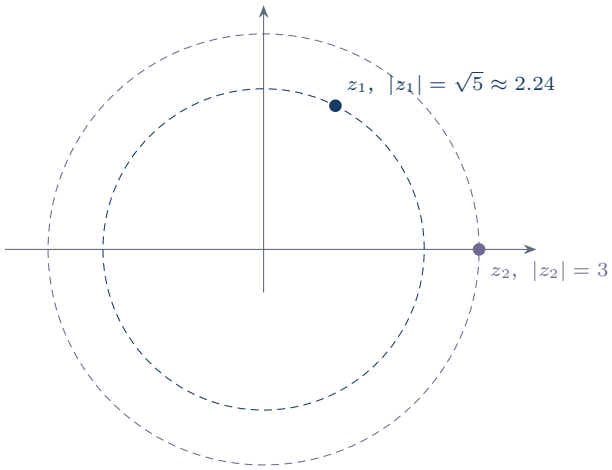


图 124　第 36 题解析图：比较两个模长（两个圆的半径）是可行的

A `T=complex<double>`，没有 < ✘

B `T=double`，比较 2.236 与 3，结果 3 ✔

C 显式写出 `complex`，仍然没有 < ✘

D `z1` 与 `3.0` 推出 `T` 冲突（`complex` 与 `double`）✘

图 125　第 36 题选项图

第 37 题　答案：ABC　难度：跨学科・中等偏下

当前专题知识：K3 实例化（编译期、按类型各生成一份）+ K5（`int` 截断）。**外部情境**：一阶数字低通滤波器。**为什么不算超纲**：更新公式已在题干给出，只需代入计算。**迁移点**：控制与信号处理中“同一算法用不同数值精度实现”正是“同一模板用不同 `T` 实例化”。

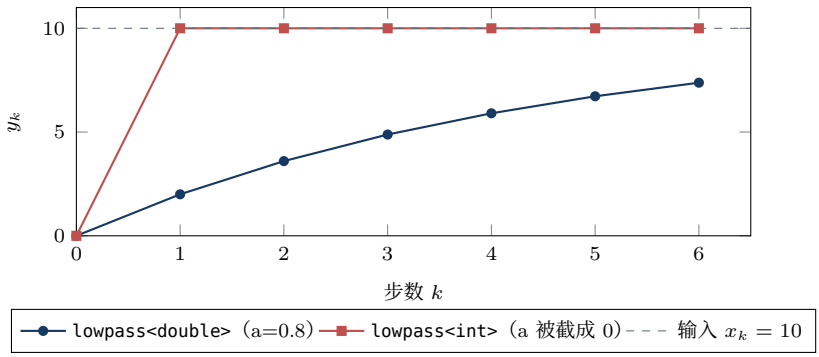


图 126　第 37 题解析图：输入从 0 突然变为 10 时两种实例的输出（数据已实测）

逐项：

A ✔：$0.8 \times 0 + 0.2 \times 10 = 2$（浮点计算得 `1.9999999999999996`，默认输出显示 `2`，故说“约为 2”），对应图中蓝线第 1 点。

B ✔：`T=int` 时实参 `0.8` 转为 `int` 得 `0`，于是 $y = 0 \cdot y_{k-1} + 1 \cdot x_k = x_k$，输出直接等于输入，红线一步跳到 10，“平滑”作用消失。

C ✔：`float` 与 `double` 是不同类型组合，各生成一个实例（K3 事实②）。

D ✘：`T` 必须在编译期确定（K3、2.2 节对照中的“✘”），不能在运行时切换；要支持两种精度，需在编译时两种实例都生成，运行时选择调用哪一个。

第 38 题　答案：13；13.5；13.5　难度：跨学科 · 基础

当前专题知识：K4 推导 + K5（同一操作 `/` 在不同类型下含义不同）。**外部情境**：动能公式。**为什么不算超纲**：公式已给出。**迁移点**：物理公式 $\frac{1}{2}mv^2$ 中的“除以 2”，在 `T=int` 的实例里变成了整数除法。

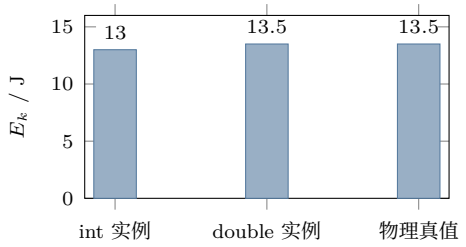


图 127　第 38 题解析图：整数实例丢失了 0.5 J

过程：`kinetic(3,3)` 推出 `T=int`，$3 \times 3 \times 3 = 27$，`27/2` 为整数除法得 `13`；`kinetic(3.0,3.0)` 推出 `T=double`，得 `13.5`；物理真值 $E_k = \frac{1}{2} \times 3 \times 3^2 = 13.5$ J（均已实测）。　**一句话**：给物理量选 `T` 时，要用能表示小数的类型。

第 9 章　总复习与参考资料

9.1　高频易错点总表

编号	错误想法 / 写法	正确理解	位置
1	函数重载就能消除重复代码	重载只统一名字，函数体仍要写多份	3.1
2	注释函数时留下 `template` 行	`template` 只管下一个声明，会“接”到 `main` 上	3.2、题 9
3	`template <T>`	必须写 `typename/class`	3.2、题 2
4	模板在运行时决定类型	实例化在编译期	3.3、题 4
5	显式指定后引用形参也能转换	引用必须绑定同类型变量	3.4、题 29
6	实例化成功 = 结果正确	可能静默截断（`c = 1.1` → 1）	3.5、题 26
7	省略类模板的尖括号	C++14 报错，C++17 推出 `const char*`	4.1、题 8
8	两个“实例化”是一回事	模板→类（编译期）；类→对象（运行期）	4.1
9	形参写 `Person &p`	`Person` 不是类型	4.2、题 14
10	类外实现只写 `Person::`	要写 `template<…>` 与 `Person<T1,T2>::`	4.3、题 9
11	所有错误都能等到实例化再报	与 `T` 无关的错误立即报	4.4

编号	错误想法 / 写法	正确理解	位置
12	`#pragma once` 能解决链接错误	链接错误源于“定义与使用不在同一编译单元”	5.2、题 15

9.2　基础解题步骤总结

遇到模板题，按这四步想

第 1 步，确定 `T`：有尖括号就按顺序取；没有就把形参与实参逐位对齐，检查是否一致、是否可推。

第 2 步，列出要求：读函数体（或被调用的成员函数），写出 `T` 需要支持的操作。

第 3 步，判断生成什么：数不同的类型组合；类模板只数被用到的成员函数。

第 4 步，检查定义在哪：使用处所在的编译单元能否看到定义；看不到就会 `undefined reference`。

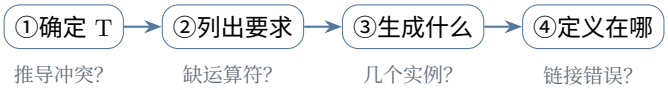


图 128　模板题四步法及每一步对应的典型错误

9.3　相近概念总对比

一对概念	关键区别	一句话
函数重载 / 函数模板	手写多份 / 生成多份	逻辑不同用重载，逻辑相同用模板
`typename` / `class`	参数列表中等价；模板内部指明类型时只能用 `typename`	参数列表里随便用
实例化 / 调用	编译期生成代码 / 运行期执行代码	先有实例，后有调用
自动推导 / 显式指定	编译器看实参 / 程序员写尖括号	推不出或要转换时显式写
函数模板 / 类模板	一步到函数 / 两步到对象；成员按需生成	类模板初学请写全尖括号
类模板 / 模板类	图纸 / 用图纸造出的类	`Person` 与 `Person<string,int>`
隐式实例化 / 显式实例化	用到时自动 / 程序员要求	显式实例化可解决分文件问题

9.4　跨学科 / 相似知识总对比表

本专题知识	相似知识（课程）	类型	能搬过来的 / 不能搬过来的
K1 函数模板	数域 P 上的定理（高等代数）	思想重现	✔写一次、具体化多次；✘实例是不同函数，不是同一个定理
K2 两层参数	函数族 $f_a(x)$（高等数学）	类比同构	✔先定外层再代内层；✘`T` 不能连续变化、不能运行时决定
K3 实例化	`#define` 宏（C 语言）	方法互补	✔都能少写代码；✘宏无类型检查、每次展开
K4 类型推导	待定系数法（高中 / 高数）	类比同构	✔对齐列方程、矛盾即无解；✘不能“近似”凑解
K5 类型要求	有序域 / 复数无序（高代、复变）	推广	△比大小只对有 `<` 的类型成立；✘`complex` 不能 `myMax`
K6 类模板	分布族 $N(\mu,\sigma^2)$（数理统计）	类比同构	✔三层结构；✘类型参数不能被“估计”
K7 参数类型	函数定义域(数学)	类比同构	✔域外不能代入；△模板化 ≈ 延拓，须满足要求
K8 类外实现	普通类类外实现（封装专题）	推广	△多写 `template` 行与 `<T1,T2>`
K9 用到才实例化	Python 运行期报错（程序设计）	同名异义	△都会“延迟报错”；✘时机（编译期 / 运行期）不同
K10 分文件	普通函数分文件（封装专题）	方法互补	✘“`.h` 放声明、`.cpp` 放定义”对模板失效

9.5　后续课程连接

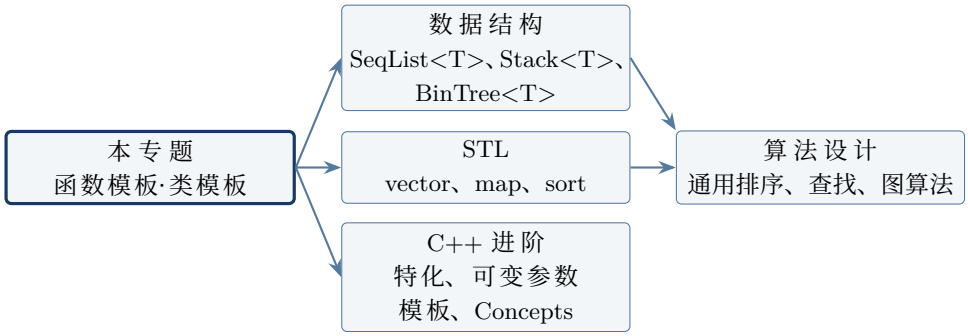


图 129　本专题之后会在哪里再次出现

最直接的下一步是数据结构课：几乎每一个容器都会以类模板的形式出现，你在第 5 章写的 `MyStack<T>` 就是它们的原型。之后在 STL 中，你会用 `std::sort(v.begin(), v.end())` 这样的函数模板，它对元素类型的要求（能用 `<` 比较）正是 3.5 节的内容。

9.6　参考资料

原始教学依据

- 本人课程练习源文件：`函数模板.cpp`、`类模板.cpp`、`类模板成员函数类外实现.cpp`、`类模板分文件编写/person.h`、`类模板分文件编写/类模板分文件.cpp`。

核心定义与规则核验

- cppreference.com: Class template（隐式实例化规则、显式实例化语法）. `https://en.cppreference.com/w/cpp/language/class_template`
- cppreference.com: Function template；Template argument deduction；Class template argument deduction (CTAD). `https://en.cppreference.com/w/cpp/language/function_template`
- ISO/IEC 14882（C++ 标准）关于模板、实例化与 `main` 函数的规定。
- 所有代码行为：GNU g++ 13（`-std=c++14 / c++17`）实测，`nm -C` 查看符号表。

学科史资料

- B. Stroustrup, *Parameterized Types for C++*, USENIX C++ Conference, 1988. `https://www.stroustrup.com/parameterized_types_1988.pdf`
- Cfront Release 3.0 源码与说明（Computer History Museum 软件保存项目）. `https://softwarepreservation.computerhistory.org/c_plus_plus/cfront/release_3.0.3/source/README3.0`
- History of the Standard Template Library（Musser、Ada、Eiffel 与 1993–1994 年 STL 进入标准的时间节点）. `https://en.wikipedia.org/wiki/History_of_the_Standard_Template_Library`
- H. Sutter, Trip Report: March 2010 ISO C++ Standards Meeting（移除 `export` 模板）；WG21 文献 N3065. `https://open-std.org/JTC1/SC22/WG21/docs/papers/2010/n3065.html`

——全文完——